

The full-stack development is enough. However, if you want to integrate AWS security monitoring or your very own EC2 to monitor, there might be changes to all instances and services (e.g., reverse proxy, honeypot, private server, S3, and DB).

Start

About CloudWatch

Source: [here](#), [here](#)

Alarm Set Up: [here](#)

About CloudTrail

Source: [here](#), here

About VPC Flow Logs

The data is set to CloudWatch anyway.

CloudWatch Vs. CloudTrail

4 services/resources:

- Reverse proxy/frontend
- backend/private instance
- Database
- S3

To Do:

- ☒ From ChatGPT & Gemini, finish connecting all AWS services and resources to CloudTrail, CloudWatch, and VPC Flow Logs

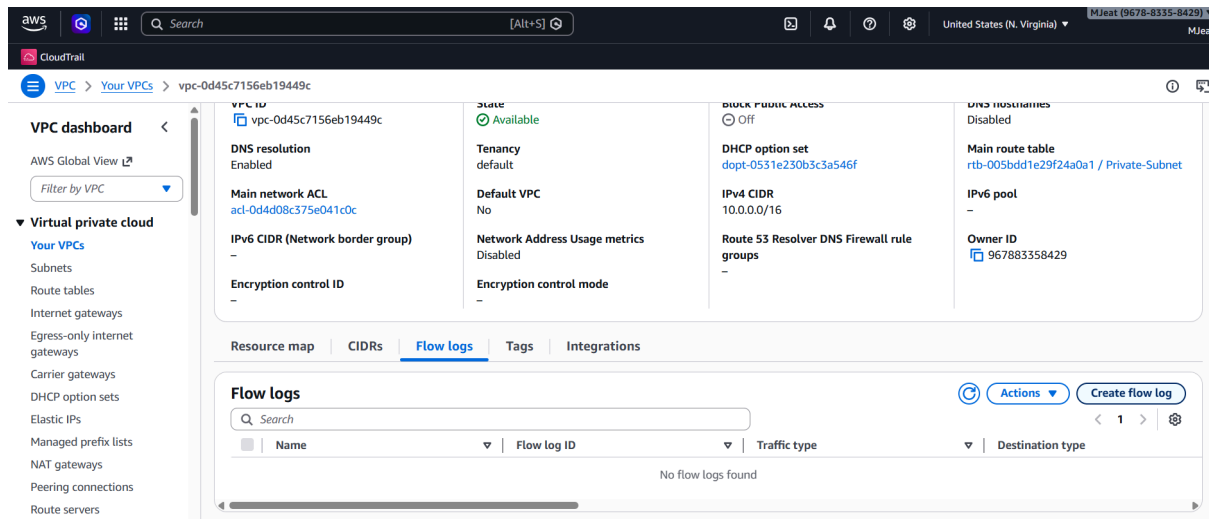
- ☑ Test if CloudTrail & CloudWatch logging and monitoring work for each AWS service
- ☑ Create a centralized SOC platform using Elastic and configure its IAM role
- ☑ Connect all logs from CloudTrail and CloudWatch to the SOC platform

=====

Connecting Resources with CloudWatch, CloudTrail, & Flow Logs

VPC Flow Logs

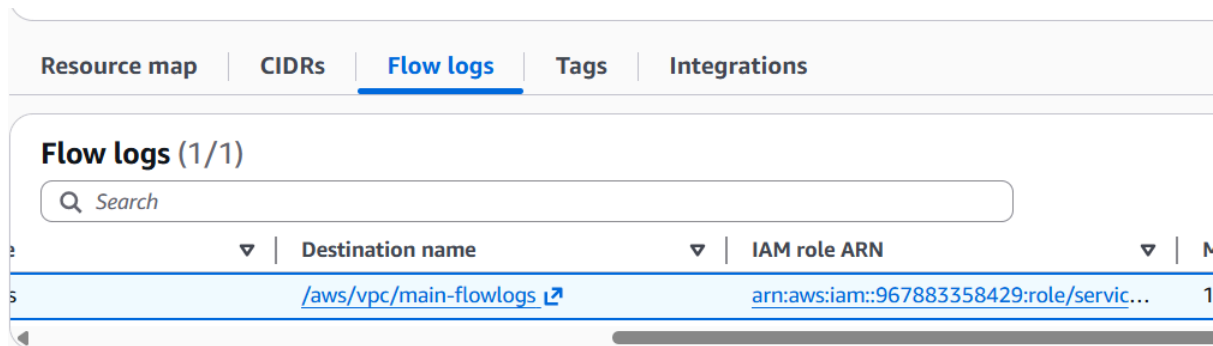
The entire log is in CloudWatch.



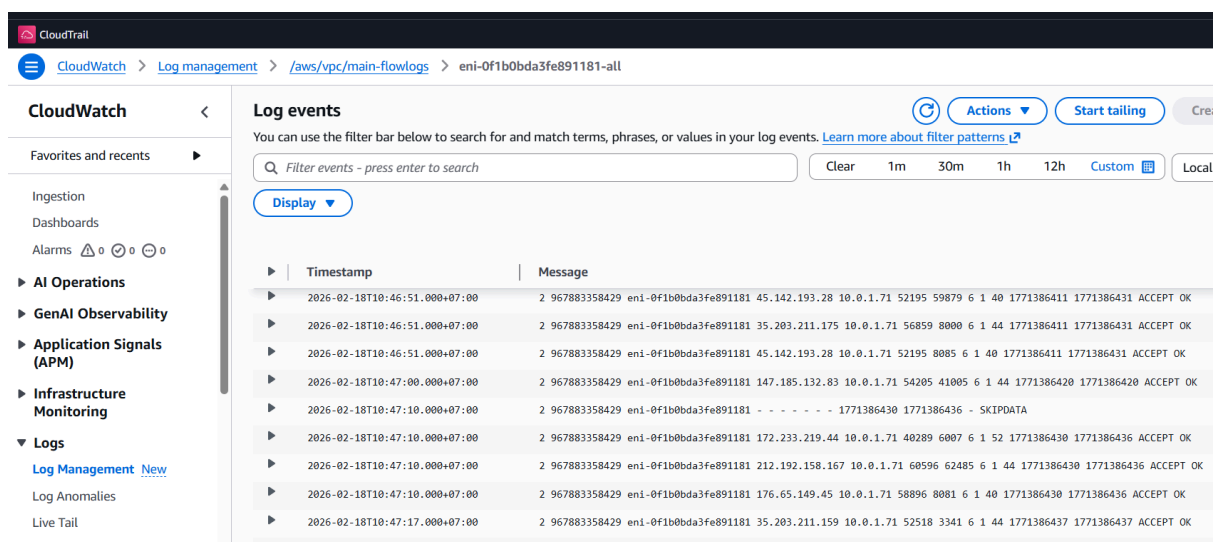
Since the Proxy, Backend, and Database all live inside the VPC subnets, one "Switch" covers them all at the network layer.

- VPC Console > Your VPCs > Select your VPC.
- Click the Flow Logs tab > Create flow log.
- Filter: ALL.
- Destination: Send to CloudWatch Logs.
- Log Group: Create /aws/vpc/main-flowlogs.
- IAM Role:
 - Click "Set up permissions." Create a role named VPCFlowLogRole.
 - Ensure the trust policy allows vpc-flow-logs.amazonaws.com to assume the role.

Confirmation:



Click the destination name link > Click the Logs Stream > If you see ACCEPT, all good.



CloudWatch

Database:

To monitor a database in a SOC, CloudTrail is not enough. CloudTrail only sees "Management" actions (like "Who deleted the database?"). To see the actual SQL queries or failed login attempts, you must use CloudWatch Logs.

Here is how you connect your MySQL Database to CloudWatch Logs:

Step 1: Enable Log Exports in RDS

1. Go to the RDS Console > Databases.
2. Select your MySQL instance and click Modify.
3. Scroll down to the Log exports section.

4. Check all four boxes: Audit log, Error log, General log, and Slow query log.
5. Click Continue at the bottom, select Apply immediately, and click Modify DB Instance.

Step 2: Configure the Database Parameters

For MySQL to actually generate those logs so they can be sent to CloudWatch, you must tell the database to turn on its "recording" feature.

1. In the RDS Console sidebar, click Parameter groups.
2. Select the Parameter Group your DB is using (if you are using the default group, you must create a "New Parameter Group" first, then associate it with your DB and modify it). Note: You have to create a new custom parameter group, not using the default ones.
3. Search for and change these parameters:
 - `general_log`: Set to 1 (This turns on the general query log).
 - `log_output`: Set to FILE (This is required for CloudWatch exports).
 - `slow_query_log`: Set to 1 (Captures slow queries for performance/security monitoring).
 - `Log_error_verbosity`: Set its value to 3 (captures logs verbosely)
4. Click Saves

☰ [Aurora and RDS](#) > [Parameter groups](#) > Create parameter group

Create parameter group

Parameter group details

Parameter group name

The name must have 1 to 255 characters and begin with a letter. The name can't end with a hyphen or contain two consecutive hyphens. Valid characters: A-Z, a-z, 0-9, and - (hyphen)

Description

This description appears in the Parameter groups dashboard. You can use it to quickly identify the purpose of the parameter group.

Engine type

Parameter group family

You can associate a DB parameter group with only one DB parameter group family. You can apply the parameter group only to a DB instance whose DB engine is compatible with the parameter group family.

Type

Type for the DB parameter group

Step 3: Attach the Group to your Database

1. Go to RDS Console > Databases > Select your DB.
2. Click Modify.
3. Scroll down to Additional configuration > Database options.
4. Change DB parameter group from default.mysql... to your new soc-mysql-parameters.

5. Scroll to the bottom, click Continue, select Apply immediately, and click Modify DB Instance.

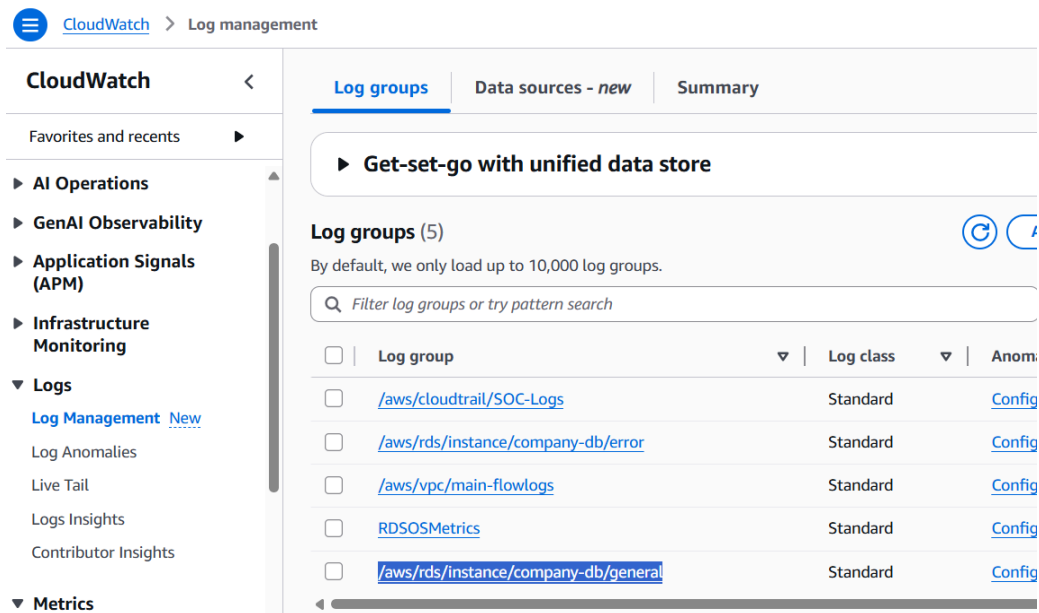
The screenshot shows the 'Modify DB instance' page in the AWS Management Console. The breadcrumb navigation at the top reads 'Aurora and RDS > Databases > Modify DB instance: company-db'. The page is divided into two main sections. The first section, 'Log exports', has the subtitle 'Select the log types to publish to Amazon CloudWatch Logs' and contains five checked checkboxes: 'Audit log', 'Error log', 'General log', 'iam-db-auth-error log', and 'Slow query log'. Below this is the 'IAM role' section, which states 'The following service-linked role is used for publishing logs to CloudWatch Logs.' and shows 'RDS service-linked role' in a light blue box. The second section, 'Additional configuration', has a dropdown arrow and the subtitle 'Database options, backup turned on, maintenance, delete protection turned off'. It contains two sub-sections: 'Database options' with a 'DB parameter group' set to 'soc-mysql-parameters-loggings' (with an 'Info' link), and 'Option group' set to 'default:mysql-8-4' (also with an 'Info' link).

Confirmation:



How to test if the Database Connection works:

1. Wait 5 minutes for the RDS change to complete.
2. Generate activity: Go to your website and perform a search or upload. This sends SQL commands to the DB.
3. Check CloudWatch: * Go to CloudWatch Console > Log groups.
 - Look for new groups named:
 - /aws/rds/instance/[your-db-name]/error
 - /aws/rds/instance/[your-db-name]/general
4. Verification: Click into the general log. If you see your SQL queries (e.g., SELECT * FROM uploads...) appearing in the log stream, the database connection is complete.



IAM Role for EC2

If you already have a private instance with an IAM role to talk with S3 (which you should already have), just attach this policy: `CloudWatchAgentServerPolicy`

Then, create a new IAM role for the frontend and only attach the same policy as well.

Install Agent for both EC2

```
sudo apt update
```

```
wget https://amazoncloudwatch-agent.s3.amazonaws.com/ubuntu/amd64/latest/amazon-cloudwatch-agent.deb
```

```
sudo dpkg -i -E ./amazon-cloudwatch-agent.deb
```

Backend / Private Instance– EC2

```
sudo nano /opt/aws/amazoncloudwatch-agent/bin/config.json
```

Then, copy and paste this code:

```
{
```

```
"logs": {  
  "logs_collected": {  
    "files": {  
      "collect_list": [  
        {  
          "file_path": "/var/log/auth.log",  
          "log_group_name": "backend-auth",  
          "log_stream_name": "{instance_id}",  
          "timezone": "UTC"  
        },  
        {  
          "file_path": "/var/log/syslog",  
          "log_group_name": "backend-syslog",  
          "log_stream_name": "{instance_id}",  
          "timezone": "UTC"  
        },  
        {  
          "file_path": "/var/log/app.log",  
          "log_group_name": "backend-app",  
          "log_stream_name": "{instance_id}",  
          "timezone": "UTC"  
        }  
      ]  
    }  
  }  
}
```

Save & Exit. Then,

```
sudo /opt/aws/amazon-cloudwatch-agent/bin/amazon-cloudwatch-agent-ctl \
```

```
-a fetch-config \
```

```
-m ec2 \
```

```
-c file:/opt/aws/amazon-cloudwatch-agent/bin/config.json \
```

```
-s
```

```
ubuntu@ip-10-0-2-37:~$ sudo /opt/aws/amazon-cloudwatch-agent/bin/amazon-cloudwatch-agent-ctl -a fetch-config -m ec2 -c file:/opt/aws/amazon-cloudwatch-agent/bin/config.json -s
***** processing amazon-cloudwatch-agent *****
Starting config-downloader, this will map back to a call to amazon-cloudwatch-agent
Executing /opt/aws/amazon-cloudwatch-agent/bin/amazon-cloudwatch-agent with arguments: [config-downloader -mode ec2 -download-source file:/opt/aws/amazon-cloudwatch-agent/bin/config.json -output-dir /opt/aws/amazon-cloudwatch-agent/etc/amazon-cloudwatch-agent.d -config /opt/aws/amazon-cloudwatch-agent/etc/common-config.toml -multi-config default]
D1 [EC2] Found active network interface
D1 imds retry client will retry 1 times
Start configuration validation...
2026/02/18 05:23:25 Reading json config file path: /opt/aws/amazon-cloudwatch-agent/etc/amazon-cloudwatch-agent.d/file_config.json.tmp ...
2026/02/18 05:23:25 I! Valid json input schema.
2026/02/18 05:23:25 Configuration validation first phase succeeded
Starting config-translator, this will map back to a call to amazon-cloudwatch-agent
Executing /opt/aws/amazon-cloudwatch-agent/bin/amazon-cloudwatch-agent with arguments: [config-translator -mode ec2 -config /opt/aws/amazon-cloudwatch-agent/etc/common-config.toml -multi-config default -input-dir /opt/aws/amazon-cloudwatch-agent/etc/amazon-cloudwatch-agent.d -output /opt/aws/amazon-cloudwatch-agent/etc/amazon-cloudwatch-agent.toml]
D1 [EC2] Found active network interface
D1 imds retry client will retry 1 times
/opt/aws/amazon-cloudwatch-agent/bin/amazon-cloudwatch-agent -schematest -config /opt/aws/amazon-cloudwatch-agent/etc/amazon-cloudwatch-agent.toml
Configuration validation second phase succeeded
Configuration validation succeeded
amazon-cloudwatch-agent has already been stopped
Created symlink /etc/systemd/system/multi-user.target.wants/amazon-cloudwatch-agent.service → /etc/systemd/system/amazon-cloudwatch-agent.service.
ubuntu@ip-10-0-2-37:~$
```

Result in CloudWatch: (Logs have been created)

The screenshot shows the AWS CloudWatch console interface. On the left, there is a navigation menu with categories like 'Log Management' and 'Metrics'. The main area displays 'Log groups (7)' with a search bar and a list of log groups. The log groups listed are:

- /aws/cloudtrail/SOC-Logs
- /aws/rds/instance/company-db/error
- /aws/vpc/main-flowlogs
- RDSOSMetrics
- /aws/rds/instance/company-db/general
- backend-auth
- backend-syslog

Frontend / Public Instance– EC2

```
sudo apt update
```

```
wget https://amazoncloudwatch-agent.s3.amazonaws.com/ubuntu/amd64/latest/amazon-cloudwatch-agent.deb
```

```
sudo dpkg -i -E ./amazon-cloudwatch-agent.deb
```

Then,

```
sudo nano /opt/aws/amazoncloudwatch-agent/bin/config.json
```

Then, copy and paste this code:

```
{  
  "logs": {  
    "logs_collected": {  
      "files": {  
        "collect_list": [  
          {  
            "file_path": "/var/log/nginx/access.log",  
            "log_group_name": "frontend-access",  
            "log_stream_name": "{frontend-instance_id}"  
          },  
          {  
            "file_path": "/var/log/nginx/error.log",  
            "log_group_name": "frontend-error",  
            "log_stream_name": "{frontend-instance_id}"  
          },  
          {  
            "file_path": "/var/log/auth.log",  
            "log_group_name": "frontend-auth",  
            "log_stream_name": "{frontend-instance_id}"  
          }  
        ]  
      }  
    }  
  }  
}
```

```
]
}
}
}
}
}
```

Save & Exit. Then,

```
sudo /opt/aws/amazon-cloudwatch-agent/bin/amazon-cloudwatch-agent-ctl \
```

```
-a fetch-config \
```

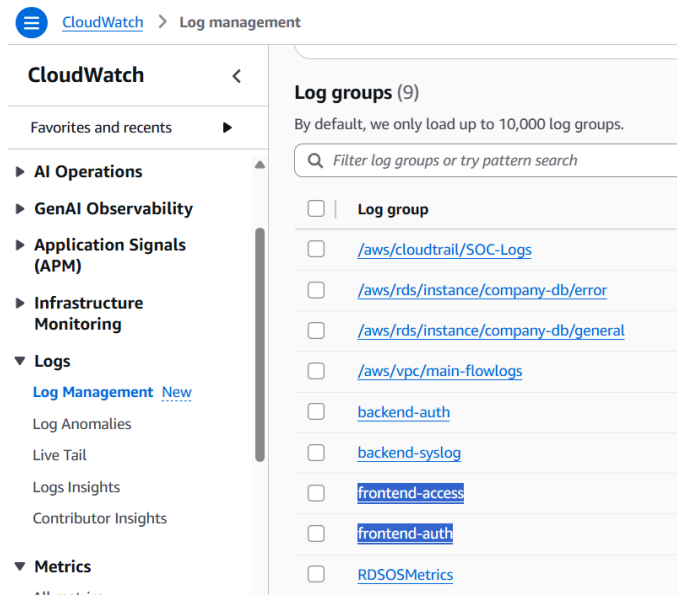
```
-m ec2 \
```

```
-c file:/opt/aws/amazon-cloudwatch-agent/bin/config.json \
```

```
-s
```

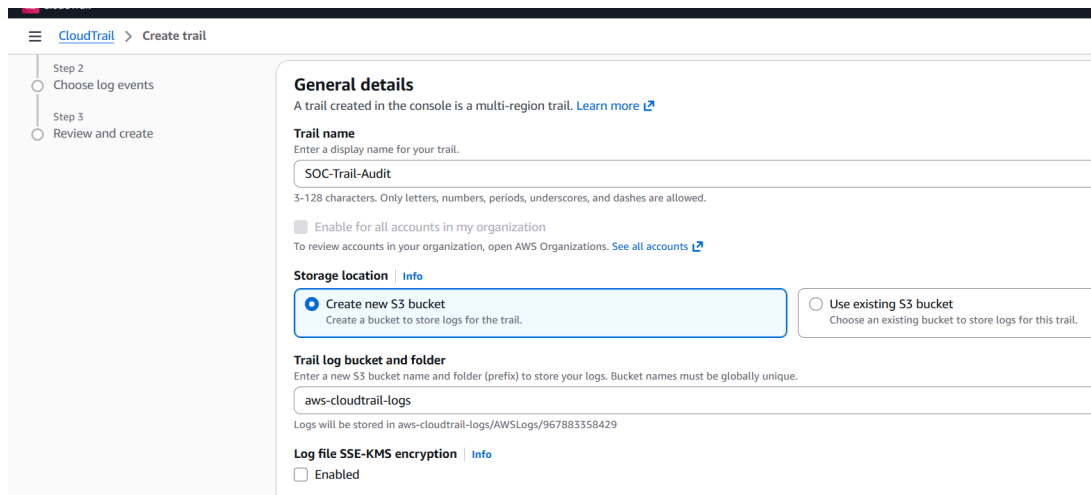
```
ubuntu@ip-10-0-1-252:~$ sudo /opt/aws/amazon-cloudwatch-agent/bin/amazon-cloudwatch-agent-ctl \
-a fetch-config \
-m ec2 \
-c file:/opt/aws/amazon-cloudwatch-agent/bin/config.json \
-s
***** processing amazon-cloudwatch-agent *****
Starting config-downloader, this will map back to a call to amazon-cloudwatch-agent
Executing /opt/aws/amazon-cloudwatch-agent/bin/amazon-cloudwatch-agent with arguments: [config-downloader
amazon-cloudwatch-agent/etc/common-config.toml -multi-config default -mode ec2 -download-source file:/opt/aws/amaz
D! [EC2] Found active network interface
I! imds retry client will retry 1 times
Start configuration validation...
2026/02/18 05:27:54 Reading json config file path: /opt/aws/amazon-cloudwatch-agent/etc/amazon-cloudwatch
2026/02/18 05:27:54 I! Valid Json input schema.
2026/02/18 05:27:54 Configuration validation first phase succeeded
Starting config-translator, this will map back to a call to amazon-cloudwatch-agent
Executing /opt/aws/amazon-cloudwatch-agent/bin/amazon-cloudwatch-agent with arguments: [config-translator
t /opt/aws/amazon-cloudwatch-agent/etc/amazon-cloudwatch-agent.json -input-dir /opt/aws/amazon-cloudwatch
ent.toml]I! Trying to detect region from ec2
D! [EC2] Found active network interface
I! imds retry client will retry 1 times
/opt/aws/amazon-cloudwatch-agent/bin/amazon-cloudwatch-agent -schematest -config /opt/aws/amazon-cloudwat
Configuration validation second phase succeeded
Configuration validation succeeded
amazon-cloudwatch-agent has already been stopped
Created symlink /etc/systemd/system/multi-user.target.wants/amazon-cloudwatch-agent.service → /etc/system
ubuntu@ip-10-0-1-252:~$ sudo systemctl start amazon-cloudwatch-agent
ubuntu@ip-10-0-1-252:~$ sudo systemctl status amazon-cloudwatch-agent
● amazon-cloudwatch-agent.service - Amazon CloudWatch Agent
   Loaded: loaded (/etc/systemd/system/amazon-cloudwatch-agent.service; enabled; preset: enabled)
   Active: active (running) since Wed 2026-02-18 05:27:55 UTC; 13s ago
     Main PID: 3031 (amazon-cloudwat)
```

Result in CloudWatch: (Logs have been created) – Final result



CloudTrail

This is for AWS account modification and S3 bucket modifications.



Note: The bucket name should be unique because it could conflict with other people's naming conventions.

- Configuration: Once you check this, you must select S3 as the resource type so it monitors your specific bucket.

3. Insights events (OPTIONAL / RECOMMENDED)

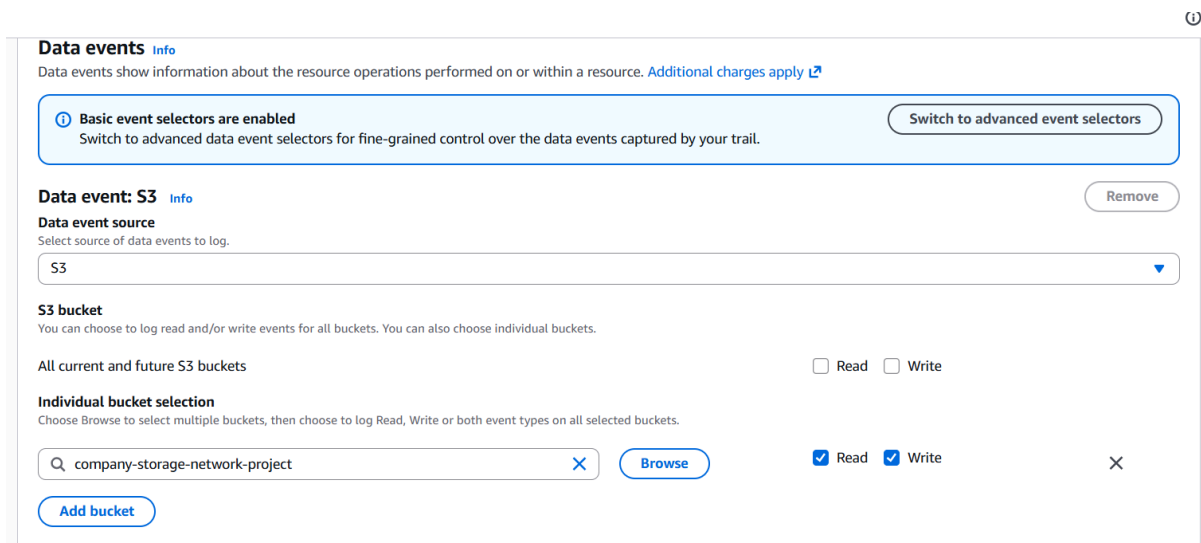
- What it is: AWS uses AI to look for "unusual" spikes in API calls. For example, if your "AdminUser" suddenly deletes 1,000 files in 1 minute, Insights will flag it as an anomaly.
- Why: Great for SOC alerts, but it takes about 7 days to "learn" your normal behavior before it starts working.
- Verdict: Check it if you want automated "Anomaly Detection."

4. Network activity events (SKIP FOR NOW)

- What it is: Logs API calls that go through VPC Endpoints.
- Why: Unless you have specifically created "Interface VPC Endpoints" for S3 or EC2 (which costs extra money per hour), you won't have any data here.
- Verdict: Uncheck it to keep your logs clean and save money.

Management Event:

Enable Read and Write. Then,



Data events [Info](#)

Data events show information about the resource operations performed on or within a resource. [Additional charges apply](#)

Basic event selectors are enabled
Switch to advanced data event selectors for fine-grained control over the data events captured by your trail. [Switch to advanced event selectors](#)

Data event: S3 [Info](#) [Remove](#)

Data event source
Select source of data events to log.
S3

S3 bucket
You can choose to log read and/or write events for all buckets. You can also choose individual buckets.

All current and future S3 buckets ☐ Read ☐ Write

Individual bucket selection
Choose Browse to select multiple buckets, then choose to log Read, Write or both event types on all selected buckets.

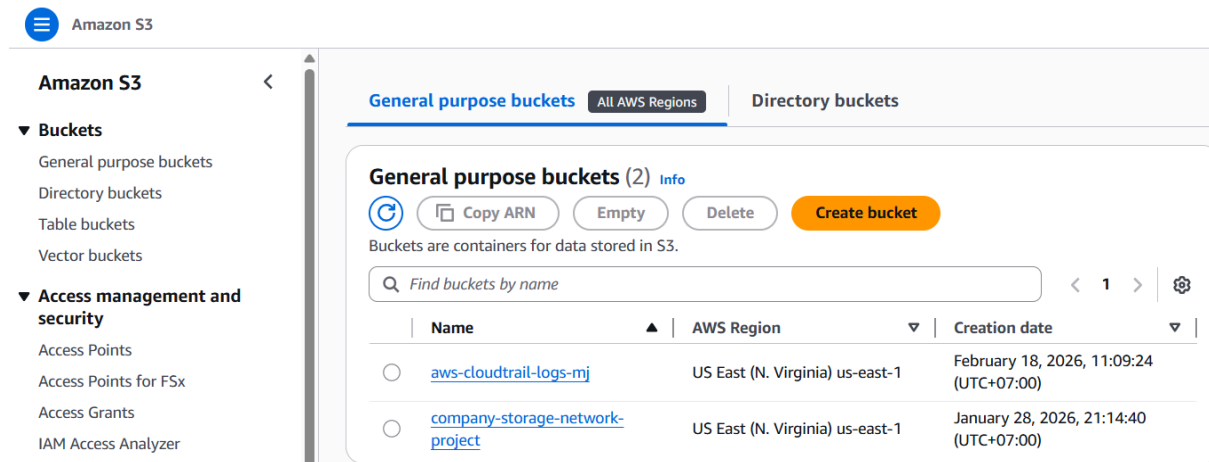
Q company-storage-network-project X [Browse](#) ☒ Read ☒ Write X

[Add bucket](#)

You can skip the Aggregated Events step. AWS recently added this to help massive enterprises (think thousands of employees) who generate millions of S3 "Read/Write" events every minute. It summarizes those thousands of events into one single "report" every 5 minutes.

Finally, click create.

Bucket Location:



Amazon S3

General purpose buckets (2) Info

Copy ARN Empty Delete Create bucket

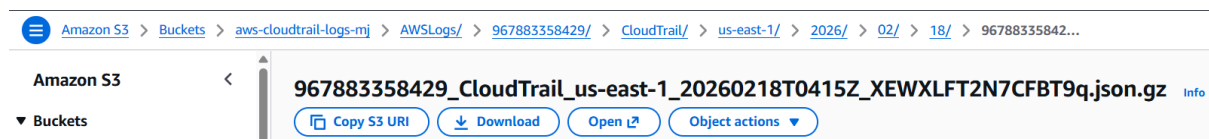
Buckets are containers for data stored in S3.

Find buckets by name

Name	AWS Region	Creation date
aws-cloudtrail-logs-mj	US East (N. Virginia) us-east-1	February 18, 2026, 11:09:24 (UTC+07:00)
company-storage-network-project	US East (N. Virginia) us-east-1	January 28, 2026, 21:14:40 (UTC+07:00)

Confirmation:

In S3, you can navigate using the path I gave. You should click the Open button and see a long JSON script.

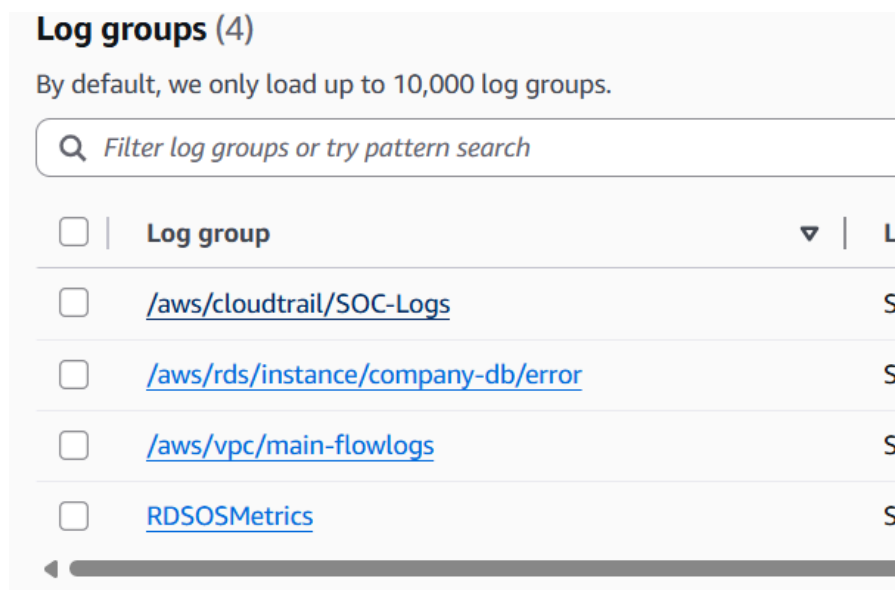


Amazon S3 > Buckets > aws-cloudtrail-logs-mj > AWSLogs/ > 967883358429/ > CloudTrail/ > us-east-1/ > 2026/ > 02/ > 18/ > 96788335842...

967883358429_CloudTrail_us-east-1_20260218T0415Z_XEWXLFT2N7CFBT9q.json.gz Info

Copy S3 URI Download Open Object actions

In CloudWatch, since you enable CloudWatch Logs while creating CloudTrail, go to CloudWatch > Log Management > Log Groups > Find your CloudTrail logs folder. You should see log streams. Check your AWS account ID with the long number in the log and check the time as well.



Log groups (4)

By default, we only load up to 10,000 log groups.

Filter log groups or try pattern search

Log group	L
☐ /aws/cloudtrail/SOC-Logs	S
☐ /aws/rds/instance/company-db/error	S
☐ /aws/vpc/main-flowlogs	S
☐ RDSOSMetrics	S

To-do List – Phase 1 & 2: Completed.

From CloudWatch

☐	Log group
☐	/aws/cloudtrail/SOC-Logs
☐	/aws/rds/instance/company-db/error
☐	/aws/rds/instance/company-db/general
☐	/aws/vpc/main-flowlogs
☐	RDSOSMetrics
☐	backend-auth
☐	backend-syslog
☐	frontend-access
☐	frontend-auth

From CloudTrail

CloudTrail

> Dashboard

CloudTrail

<

Dashboard

Event history

Insights

▼ Lake

Dashboards

Query

Event data stores

Integrations

Trails

Settings

Pricing

Documentation

Forums

FAQs

Dashboard

Info

Query results history

Choose a query to view results from the last seven days.

Create a new query

No queries

No queries to display

Trails

Info

Name

SOC-Trail-Audit

CloudTrail Insights

Info

CloudTrail Insights is not enabled

Insights are events that show unusual API activity. After you enable Insights, if unusual activity is logged, Insights events are shown in this table

Event history

Info

Event name	Event time	Event source
CreateLogStream	February 18, 2026, 12:28:01 (UTC-08:00)	logs.amazonaws.com
CreateLogStream	February 18, 2026, 12:28:01 (UTC-08:00)	logs.amazonaws.com
CreateLogGroup	February 18, 2026, 12:28:01 (UTC-08:00)	logs.amazonaws.com
CreateLogGroup	February 18, 2026, 12:28:01 (UTC-08:00)	logs.amazonaws.com

=====

Creating a Centralized SOC Platform

Based on ur opinion, where should the SOC platform be located in the real world (networking and subnets)? I want my project to be as close to the real enterprise SOC platform location as possible. Then, tell me how to set it up in AWS. I'm using the AWS free tier, so the VM selection is limited.

I want to make a new VPC dedicated to security monitoring that only allow traffics from the selected resources from the main VPC or cloudwatch, cloudtrail, and VPC flow logs. Guide me to this. I can use t2.medium, FYI. Make sure to be as realistic as possible. Guide me from making a new VPC, set it up to only allow from ... idk you tell me im noob. Then, the guide on setting up the instance for this security VPC. Then, configure this VM until finish, then we can get logs from CloudWatch, CloudTrail, and VPC flow logs.

Security VPC – Creating a new VPC

Create VPC [Info](#)

A VPC is an isolated portion of the AWS Cloud populated by AWS objects, such as Amazon

VPC settings

Resources to create [Info](#)
Create only the VPC resource or the VPC and other networking resources.

☒ VPC only ☐ VPC and other networking resources

Name tag - optional
Creates a tag with a key of 'Name' and a value that you specify.

Security-VPC

IPv4 CIDR block [Info](#)

☒ IPv4 CIDR manual input ☐ IPAM-allocated IPv4 CIDR block

IPv4 CIDR

10.1.0.0/16

CIDR block size must be between /16 and /28.

IPv6 CIDR block [Info](#)

☒ No IPv6 CIDR block ☐ IPAM-allocated IPv6 CIDR block

Make sure the IPv4 CIDR is different from your main company production VPC.

Internet Gateway – Creating a new IGW

The following internet gateway was created: igw-012521857127b5a55 - security-igw. You can now attach to a VPC to enable the VPC to communicate with the internet.

Attach to a VPC

igw-012521857127b5a55 / security-igw

Actions

Details

Info

Internet gateway ID

igw-012521857127b5a55

State

Detached

VPC ID

-

Owner

967883358429

Tags (1)

Manage tags

Search tags

< 1 >

Key	Value
Name	security-igw

Then, look at the top right corner. Attach this IGW to the security VPC. Done.

Internet gateways (3)

Info

Find internet gateways by attribute or tag

☐	Name	Internet gateway ID	State
☐	-	igw-09d483907a22bd60d	Attached
☐	Company-Internet-Gateway	igw-0276be85b146980ee	Attached
☐	security-igw	igw-012521857127b5a55	Attached

You should notice that there's only one VPC (if you have 2 VPCs). That's because other VPCs, by assumption, already have an IGW. You can't attach 1 IGW to 2 different VPCs.

Private Subnet – Creating a new Private Subnet

This subnet is for hosting our private SOC platform. Make sure to disable auto-assigned public IPv4. Can check in the Action button.

Subnet settings

Specify the CIDR blocks and Availability Zone for the subnet.

Subnet 1 of 1

Subnet name

Create a tag with a key of 'Name' and a value that you specify.

soc-private-subnet

The name can be up to 256 characters long.

Availability Zone [Info](#)

Choose the zone in which your subnet will reside, or let Amazon choose one for you.

No preference

IPv4 VPC CIDR block [Info](#)

Choose the VPC's IPv4 CIDR block for the subnet. The subnet's IPv4 CIDR must lie within this block.

10.1.0.0/16

IPv4 subnet CIDR block

10.1.1.0/24

Private Route Table– Creating a new Private Route Table

Route tables (1/7) [Info](#)

Last updated 12 minutes ago [Actions](#) [Create route table](#)

	Name	Route table ID	Explicit subnet associ...	Edge associations	Main	VPC
☐	soc-public-route-table	rtb-085cdf313b73a4866	–	–	No	vpc-05a398aeeac6964a6 Sec
☒	soc-private-route-table	rtb-0b536637518d127a3	–	–	No	vpc-05a398aeeac6964a6 Sec
☐	server-public-routable	rtb-0434e94730ae627cd	subnet-0ca040edb71ba0...	–	No	vpc-0d45c7156eb19449c VP
☐	RDS-Pvt-rt	rtb-07df80a42105d95d8	5 subnets	–	No	vpc-0d45c7156eb19449c VP
☐	Private-Subnet	rtb-005bdd1e29f24a0a1	–	–	Yes	vpc-0d45c7156eb19449c VP
☐	–	rtb-077aa526aedad6396	–	–	Yes	vpc-0323b64209f368ca0 Tes
☐	–	rtb-00f5bc725f18d9e04	–	–	Yes	vpc-05a398aeeac6964a6 Sec

Make sure it is associated with the private subnet. Go to Action > Edit Subnet Association > Select private subnet.

VPC > Route tables > rtb-0b536637518d127a3 > Edit subnet associations

Edit subnet associations

Change which subnets are associated with this route table.

Available subnets (1/2)

	Name	Subnet ID	IPv4 CIDR
☒	soc-private-subnet	subnet-04d3240c9a0d9244a	10.1.1.0/24
☐	soc-public-subnet	subnet-0e917f36f9444b8e6	10.1.2.0/24

Selected subnets

subnet-04d3240c9a0d9244a / soc-private-subnet [X](#)

Save & then Edit Routes:

☰ [VPC](#) > [Route tables](#) > [rtb-0b536637518d127a3](#) > Edit routes

Edit routes

Destination	Target	Status	Propagated
10.1.0.0/16	local	✓ Active	No
0.0.0.0/0	local		
	NAT Gateway	-	No
	nat-15bdd96449a8666b9		
	Use: "nat-15bdd96449a8666b9"		
	nat-15bdd96449a8666b9 (soc-public-subnet-NAT-gateway)		

Now traffic flows:

SOC EC2 → NAT → IGW → Internet

Public Subnet – Creating a new Public Subnet

This is for the NAT gateway to connect to the IGW for Internet access.

☰ [VPC](#) > [Subnets](#) > Create subnet

Subnet 1 of 1

Subnet name
Create a tag with a key of 'Name' and a value that you specify.

The name can be up to 256 characters long.

Availability Zone [Info](#)
Choose the zone in which your subnet will reside, or let Amazon choose one for you.

IPv4 VPC CIDR block [Info](#)
Choose the VPC's IPv4 CIDR block for the subnet. The subnet's IPv4 CIDR must lie within this block.

IPv4 subnet CIDR block

< > ^ v

Then, go to this subnet > Action > Edit subnet setting > Enable Public IPv4

Edit subnet settings [Info](#)

Subnet

Subnet ID

 subnet-0e917f36f9444b8e6

Name

 soc-public-subnet

Auto-assign IP settings [Info](#)

Enable AWS to automatically assign a public IPv4 or IPv6 address to a new primary network interface for an instance in this subnet.

☒ Enable auto-assign public IPv4 address [Info](#)

☐ Enable auto-assign customer-owned IPv4 address [Info](#)
Option disabled because no customer owned pools found.

Public Route Table– Creating a new Public Route Table

[VPC](#) > [Route tables](#) > Create route table

Create route table [Info](#)

A route table specifies how packets are forwarded between the subnets within your VPC, the internet

Route table settings

Name - optional
Create a tag with a key of 'Name' and a value that you specify.

soc-public-route-table

VPC
The VPC to use for this route table.

vpc-05a398aeeac6964a6 (Security-VPC)

Tags

A tag is a label that you assign to an AWS resource. Each tag consists of a key and an optional v

Then, edit routes:

[VPC](#) > [Route tables](#) > [rtb-085cdf313b73a4866](#) > Edit routes

Edit routes

Destination	Target	Status	Propagated	Route Origin
10.1.0.0/16	local	Active	No	CreateRouteTable
Q 0.0.0.0/0	Q local			
	Internet Gateway	-	No	CreateRoute
	Q igw-012521857127b5a55			

[Add route](#)

This would make this subnet to have access to the Internet because we added an Internet Gateway.

NAT Gateway– Creating a new NAT Gateway

This is to place this gateway in the public subnet.

VPC > NAT gateways > Create NAT gateway

Name - optional
Create a tag with a key of 'Name' and a value that you specify.

The name can be up to 256 characters long.

Availability mode [Info](#)
Choose whether to deploy across all zones in the region or restrict to a single availability zone.

☒ **Regional - new**
Scales automatically across all regional AZs, simplifying management for multi AZ deployments.

☐ **Zonal**
Provides granular control within a specific availability zone, adhering to subnet level settings.

VPC
Select a VPC in which to create the regional NAT gateway.

vpc-05a398aeeac6964a6 (Security-VPC)

Connectivity type
Select a connectivity type for the NAT gateway.

☒ **Public**

☐ Private

Method of Elastic IP (EIP) allocation [Info](#)
Choose how IP addresses are associated with NAT gateways.

☒ **Automatic**
AWS automatically manages EIPs and AZ coverage for NAT gateways. This ensures easy scaling-- adding AZs automatically allocates EIPs, simplifying management.

☐ **Manual**
Manually assigns specific IP addresses for compliance or whitelisting. Note: Requires manual scaling to new AZs as workloads expand.

Then, wait until the NAT gateway finishes creating. Then, you should check its Elastic IP. It should be automatically created for the NAT gateway. Just rename it.

Elastic IP addresses (1/3) [Info](#)

Actions

Allocate Elastic IP address

	Name	Allocated IPv4 addr...	Type	Allocation ID	Reverse DNS record	A:
☐	Company-NAT-Gateway-elas-IP	3.89.52.3	Public IP	eipalloc-078b6e7ff5eeb68e2	–	–
☐	n8n-Instance-Elastic-IP	34.227.211.40	Public IP	eipalloc-0aea1f6c8de15f4fe	–	i-
☒	Security-NAT-Gateway-elas-IP	54.163.123.104	Public IP	eipalloc-01f30b8e39a1ee9c9	–	–

Private Instance – Creating a private instance for the SOC platform

Instance state: **Running**

Start instance

	Name	Instance ID
☐	n8n-Instance	i-Od010ef71b5ac0cdb
☒	soc-platform	i-035d4cb1342acd867
☐	Load-Balancer	i-03035f16a3cd073d6
☐	Private-Server	i-0cb6dfdc31bfb74e0

Private IP: 10.1.1.25

No Public IP. If you see, remove it.

For security group rules:

Outbound

Allow all types, source: 0.0.0.0/0

WARNING (LAB ONLY): In a "Military Grade" enterprise, they might restrict outbound traffic (Egress) to only specific AWS IP ranges to prevent "Data Exfiltration" (a hacker sending your data to their own server).

However, for the project:

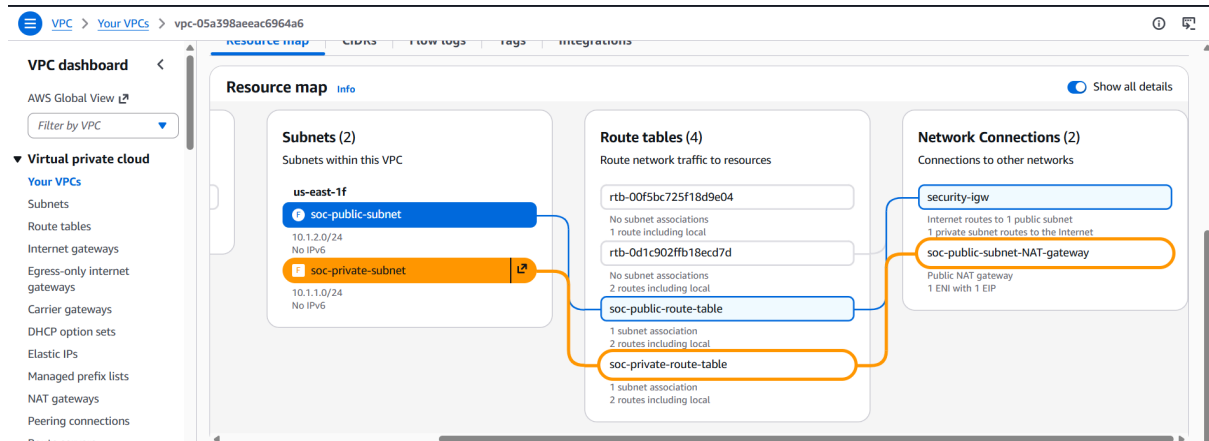
- The Elastic Installer: Needs to reach the internet to download gigabytes of software.
- AWS APIs: The instance needs to reach the global CloudWatch and S3 endpoints to pull your logs.
- Verdict: Use one rule: All Traffic | All Ports | Destination: 0.0.0.0/0. This is the standard "default" for a reason; it ensures your tools don't break because they can't "call home."

Inbound:

Type	Port Range	Source	Purpose
Custom TCP	9200	SOC-SG	Allows the SOC agent to talk to the SOC database
Custom TCP	5601	Your-Home-IP/32 or /24	Kibana UI. Accessing the dashboard from your browser.
Custom TCP	9200	10.0.0.0/16	Elasticsearch API. Allows your Main VPC instances (Frontend/Backend) to send logs.
Custom TCP	8220	10.0.0.0/16	Fleet Server. Required if you use Elastic Agents to manage logs from your other servers.

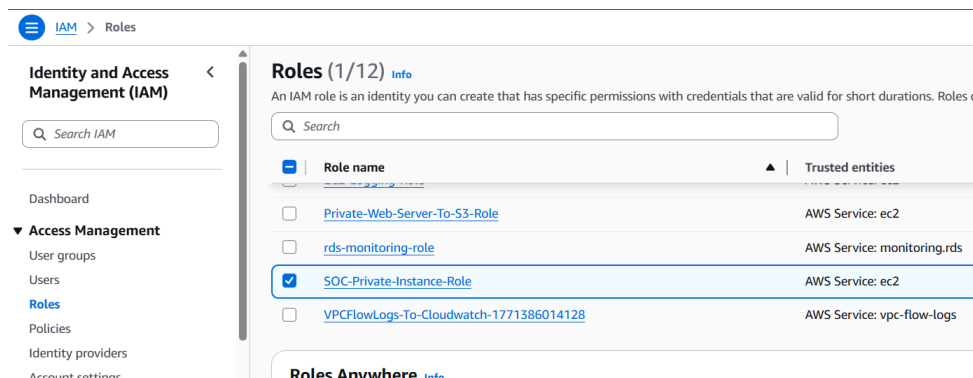
Security VPC – Resource Map Guide

It should look like this



IAM Role Set Up

We need to create an IAM role.



Trusted entity:

AWS Service

Use case:

EC2

Attach these policies:

AmazonSSMMManagedInstanceCore

CloudWatchReadOnlyAccess

AmazonS3ReadOnlyAccess

AWSCloudTrail_ReadOnlyAccess

Name:

SOC-Private-Instance-Role

Then, create a role. Next, attach it to the SOC instance.

Attach to Instance: EC2 Console > Select SOC Instance > Actions > Security > Modify IAM Role > Select **SOC-Private-Instance-Role**

SSM – AWS System Manager

Source: [here](#)

Since our SOC instance is in the private subnet with no public IP, we can't SSH from our PC. Thus, we use SSM. You can search for System Manager > Session Manager in the AWS search bar.

✅ **BEST METHOD: AWS Session Manager (Enterprise Standard)**

Do **NOT**:

- ❌ Add public IP
- ❌ Create bastion host (cost + complexity)
- ❌ Open SSH to 0.0.0.0/0

Real enterprises use:

🔒 AWS Systems Manager (SSM) Session Manager

It allows:

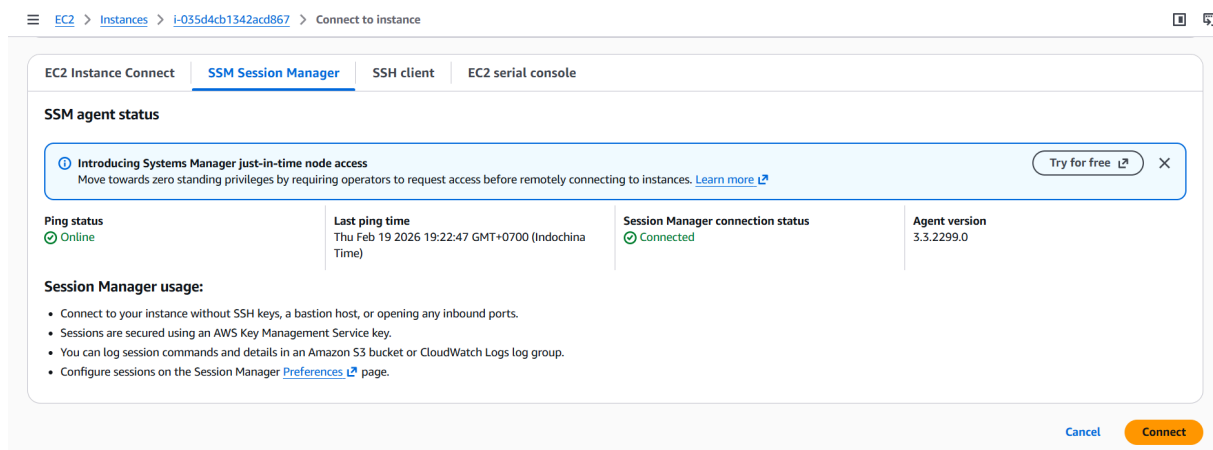
- Shell access
- No public IP
- No inbound ports needed
- Fully logged access

We will use this.

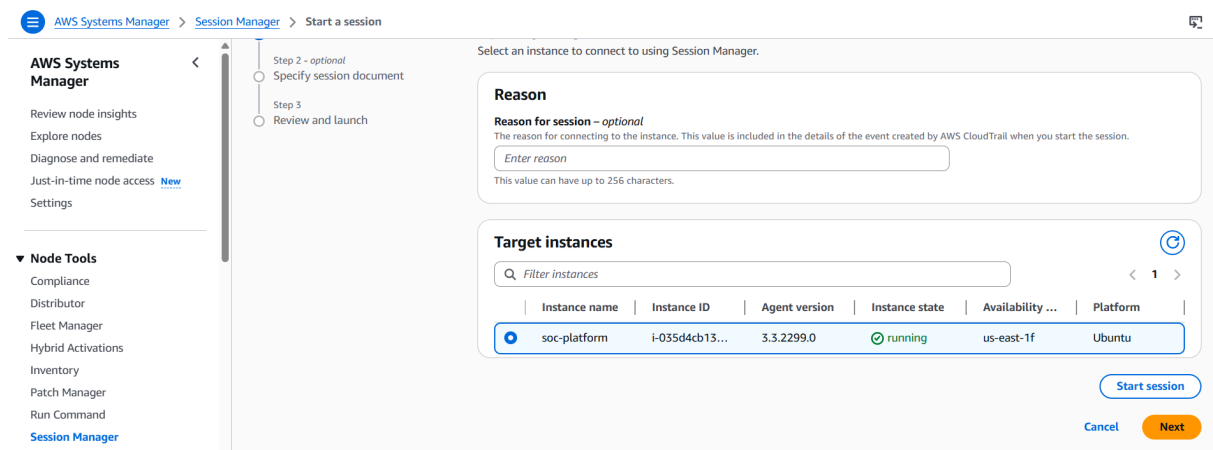
How to "SSH" without an IP

1. Go to the EC2 Console > Instances.
2. Select your SOC Platform instance.
3. Click the Connect button at the top.
4. Choose the Session Manager tab and click **Connect**.
 - Note: If the "Connect" button is greyed out, wait 5 minutes for the SSM Agent (pre-installed on Ubuntu) to check in with the new IAM role.
5. A terminal will open in your browser. You are now "inside" your private SOC instance!

From the SOC instance:



From Session Manager:



After you click **Connect** in the EC2 console or **Start Session** from the Session Manager, you should see a browser pop up and show this:

Session ID: root-i28ud49bb3cs3bjjioiprts74 [Shortcuts](#) Instance ID: i-035d4cb1342acd867 [Terminate](#)

```
$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: enX0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 9001 qdisc mq state UP group default qlen 1000
    link/ether 16:ff:da:42:f2:f1 brd ff:ff:ff:ff:ff:ff
    inet 10.1.1.25/24 metric 100 brd 10.1.1.255 scope global dynamic enX0
        valid_lft 2407sec preferred_lft 3407sec
    inet6 fe80::14ff:daff:fe42:f2f1/64 scope link
        valid_lft forever preferred_lft forever
$
```

Ping connection test:

Session ID: root-i28ud49bb3cs3bjjioiprts74 [Shortcuts](#) Instance ID: i-035d4cb1342acd867

```
$ ping google.com
PING google.com (192.178.155.102) 56(84) bytes of data.
64 bytes from yuiadrs-in-f102.1e100.net (192.178.155.102): icmp_seq=1 ttl=105 time=3.27 ms
64 bytes from yuiadrs-in-f102.1e100.net (192.178.155.102): icmp_seq=2 ttl=105 time=2.74 ms
64 bytes from yuiadrs-in-f102.1e100.net (192.178.155.102): icmp_seq=3 ttl=105 time=2.55 ms
^C
--- google.com ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2004ms
rtt min/avg/max/mdev = 2.552/2.853/3.272/0.305 ms
$
```

The downside is that you can't scroll up to the previous command, nor can you copy and paste any code or CLI inside. You need to type manually.

VPC Peering/Tunneling Connection

Source: [here](#)

For the main VPC to connect to the security VPC, each VPC's route table has to have each other's IP address.

10.0.0.0/16 – Main VPC

10.1.0.0/16 – Security VPC

First, test the connection using an instance from the public subnet of any of the VPCs and curl another public instance of another VPC. See if you can curl it.

Configuration & setup for both EC2 instances:

Instance from the main VPC – public subnet:

```
[ec2-user@ip-10-0-1-34 ~]$ sudo ls /var/www/
cgi-bin/  html/
[ec2-user@ip-10-0-1-34 ~]$ sudo ls /var/www/html/index.html
/var/www/html/index.html
[ec2-user@ip-10-0-1-34 ~]$ curl -I 10.1.2.65
curl: (28) Failed to connect to 10.1.2.65 port 80 after 129394 ms: Could not connect to server
[ec2-user@ip-10-0-1-34 ~]$
```

Instance from the Security VPC – public subnet:

{I deleted the image, but it resulted in the same Failed to connect error as shown in the above image}

Second, create a VPC peering connection.

Creating Peering Connection

VPC > Peering connections > Create peering connection

Peering connection settings

Name - *optional*

Create a tag with a key of 'Name' and a value that you specify.

Main-to-Security-PeerConnection

Select a local VPC to peer with

VPC ID (Requester)

vpc-0d45c7156eb19449c (VPC-Company-Production-Network)

VPC CIDRs for vpc-0d45c7156eb19449c (VPC-Company-Production-Network)

CIDR	Status	Status reason
10.0.0.0/16	✔ Associated	-

Select another VPC to peer with

Account

☒ My account

☐ Another account

Region

☒ This Region (us-east-1)

☐ Another Region

VPC ID (Accepter)

vpc-05a398aeaac6964a6 (Security-VPC) ▼

VPC CIDRs for vpc-05a398aeaac6964a6 (Security-VPC)

CIDR	Status	Status reason
10.1.0.0/16	✔ Associated	-

Tags

A tag is a label that you assign to an AWS resource. Each tag consists of a key and an optional value. You can use tags to search and filter your resources or t

Key **Value - optional**

Q Name X Q Main-to-Security-PeerConnection

[Add new tag](#)

You can add 49 more tags.

Note: Any VPC can be an acceptor/requester or the other way around. It doesn't matter. It's just a Point-to-point connection.

Accept Request

🟢 A VPC peering connection pcx-02b915c0f614d2097 / Main-to-Security-PeerConnection has been requested.

pcx-02b915c0f614d2097 / Main-to-Security-PeerConnection [Actions](#)

ⓘ Pending acceptance
You can accept or reject this peering connection request using the 'Actions' menu. You have until Friday, February 27, 2026 at 21:32:26 GMT+7 to accept otherwise it expires.

Details Info

Requester owner ID
967883358429

Peering connection ID
pcx-02b915c0f614d2097

Status
⌚ Pending Acceptance by 967883358429

Expiration time
Friday, February 27, 2026 at 21:32:26 GMT+7

Accepter owner ID
967883358429

Requester VPC
vpc-0d45c7156eb19449c / VPC-Company-Production-Network

Requester CIDRs
10.0.0.0/16

Requester Region
N. Virginia (us-east-1)

VPC Peering connection A...
arn:aws:ec2:us-east-1:967883358429:vpc-peering-con-
nection/pcx-02b915c0f614d2097

Accepter VPC
vpc-05a398aeaac6964a6 / Security-VPC

Accepter CIDRs
-

Accepter Region
N. Virginia (us-east-1)

[DNS](#) | [Route tables](#) | [Tags](#)

Peering Routable

Next, go to the route table for each VPC and add the IP range of the other VPC. For the main VPC to connect to the security VPC, each VPC's route table has to have each other's IP address.

Route tables (4/8) Info Last updated less than a minute ago

Q Find route tables by attribute or tag

	Name	Route table ID	Expli...	Edge associa...	Main	VPC
☐	soc-public-subnet-NAT-gateway	rtb-0d1c902ffb18ecd7d	-	nat-15bdd9644...	No	vpc-05a398aeaa
☒	soc-public-route-table	rtb-085cdf313b73a4866	subnet-...	-	No	vpc-05a398aeaa
☒	soc-private-route-table	rtb-0b536637518d127a3	subnet-...	-	No	vpc-05a398aeaa
☒	server-public-route-table	rtb-0434e94730ae627cd	subnet-...	-	No	vpc-0d45c7156e
☒	server-private-route-table	rtb-005bdd1e29f24a0a1	-	-	Yes	vpc-0d45c7156e
☐	RDS-Pvt-rt	rtb-07df80a42105d95d8	5 subnets	-	No	vpc-0d45c7156e
☐	-	rtb-00f5bc725f18d9e04	-	-	Yes	vpc-05a398aeaa
☐	-	rtb-077aa526aedad6396	-	-	Yes	vpc-0323b64205

Both SOC's route tables (public & private) can follow the same setup from the public routable below. Add the main VPC's IP inside:

Destination	Target	Status	Propagated	Route Origin
10.1.0.0/16	local	Active	No	CreateRouteTable
0.0.0.0/0	Internet Gateway	Active	No	CreateRoute
10.0.0.0/16	Peering Connection	-	No	CreateRoute

Both main VPCs' route tables (public & private) can follow the same setup from the public routable below. Add the main VPC's IP inside:

Testing if the VPC peering connection works:

Instance from the Main VPC – public subnet:

```
Last login: Fri Feb 20 14:19:22 2026 from 18.206.107.29
[ec2-user@ip-10-0-1-34 ~]$ curl -I 10.1.2.65
HTTP/1.1 200 OK
Date: Fri, 20 Feb 2026 15:19:49 GMT
Server: Apache/2.4.66 (Amazon Linux)
Last-Modified: Fri, 20 Feb 2026 14:17:26 GMT
ETag: "32-64b4214a7a160"
Accept-Ranges: bytes
Content-Length: 50
Content-Type: text/html; charset=UTF-8

[ec2-user@ip-10-0-1-34 ~]$ curl 10.1.2.65
<h1>Hello. This is ip-10-1-2-65.ec2.internal</h1>
[ec2-user@ip-10-0-1-34 ~]$
```

Instance from the Security VPC – public subnet:

```
Last login: Fri Feb 20 15:18:39 2026 from 18.206.107.27
[ec2-user@ip-10-0-1-34 ~]$ curl -I 10.1.2.65
HTTP/1.1 200 OK
Date: Fri, 20 Feb 2026 15:21:58 GMT
Server: Apache/2.4.66 (Amazon Linux)
Last-Modified: Fri, 20 Feb 2026 14:17:26 GMT
ETag: "32-64b4214a7a160"
Accept-Ranges: bytes
Content-Length: 50
Content-Type: text/html; charset=UTF-8

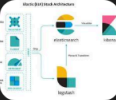
[ec2-user@ip-10-0-1-34 ~]$ curl 10.1.2.65
<h1>Hello. This is ip-10-1-2-65.ec2.internal</h1>
[ec2-user@ip-10-0-1-34 ~]$
```

Elastic Setup

Set up Elastic on the instance in the private subnet of the Security VPC.

Source (didn't use): [here](#)

The ELK Stack (Elasticsearch, Logstash, Kibana) is a powerful, open-source centralized logging and security analytics platform used in cybersecurity to ingest, store, and visualize massive volumes of log data. It acts as a SIEM (Security Information and Event Management) tool, allowing security teams to detect, investigate, and respond to threats in real-time.  YouTube +4



Core Components of the ELK Stack:

-  **Elasticsearch (E):** A distributed, RESTful search and analytics engine that stores and indexes log data in JSON format.  Medium +2
-  **Logstash (L):** A data processing pipeline that ingests, filters, transforms, and ships logs from various sources into Elasticsearch.  Medium +1
-  **Kibana (K):** A visualization dashboard that allows users to create graphs, maps, and dashboards to analyze data and monitor security events.  YouTube +4

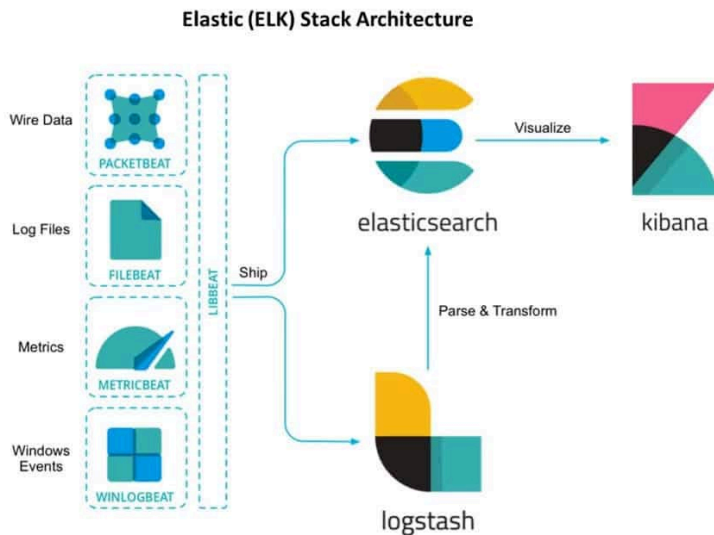


Image Source: [here](#)

Initialization

First, get into the SOC platform via SSM. Then follow the CLIs below.

```
sudo apt-get update && sudo apt-get upgrade -y
```

```
sudo apt install openjdk-21-jdk (Note that 21 is the version. Stay updated)
```

Install Docker

```
sudo apt-get install -y docker.io
```

Start Docker and ensure it runs on boot

```
sudo systemctl start docker
```

```
sudo systemctl enable docker
```

Switch to root

```
sudo -i
```

Add ssm-user to the docker group

```
usermod -aG docker ssm-user
```

Verify

```
id ssm-user # Should show: groups=...,docker
```

```
docker ps # Should return an empty list, not a permission error
```

Installation

```
cd ~
```

```
curl -fsSL https://elastic.co/start-local | sh
```

This will spin up Elasticsearch + Kibana via Docker Compose and print out something like:

 Kibana is up!

URL: http://localhost:5601

Username: elastic

Password: <SAVE_THIS_PASSWORD>

Elastic Credentials

```
i Do not use this script in a production environment

🔧 Setting up Elasticsearch and Kibana v9.3.0...

- Generated random passwords
- Created the elastic-start-local folder containing the files:
  - .env, with settings
  - docker-compose.yml, for Docker services
  - start/stop/uninstall commands
- Running docker compose up --wait

[+] up 30/30
✓ Image docker.elastic.co/elasticsearch/elasticsearch:9.3.0 Pulled
✓ Image docker.elastic.co/kibana/kibana:9.3.0 Pulled
✓ Network elastic-start-local_default Created
✓ Volume elastic-start-local_dev-kibana Created
✓ Volume elastic-start-local_dev-elasticsearch Created
✓ Container es-local-dev Healthy
✓ Container kibana-local-settings Exited
✓ Container kibana-local-dev Healthy

🎉 Congrats, Elasticsearch and Kibana are installed and running in Docker!

🌐 Open your browser at http://localhost:5601

Username: elastic
Password: jZQ7Ve7F

🔧 Elasticsearch API endpoint: http://localhost:9200
🔑 API key: X3pZY2Zwd0JBT3IwNFQtcVoyWUc6QUxfVUV2OFBOZlVqVnhPWFdXcTl4UQ==

Learn more at https://github.com/elastic/start-local

$
```

🎉 Congrats, Elasticsearch and Kibana are installed and running in Docker!

🌐 Open your browser at http://localhost:5601

Username: elastic

Password: jZQ7Ve7F

🔧 Elasticsearch API endpoint: http://localhost:9200

🔑 API key:
X3pZY2Zwd0JBT3IwNFQtcVoyWUc6QUxfVUV2OFBOZlVqVnhPWFdXcTl4UQ==

Check if both containers are healthy and active:

`docker ps`

Should show elasticsearch and kibana containers with status "healthy"

```
$ docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS
3b1f43fecbb1   docker.elastic.co/kibana/kibana:9.3.0  "/bin/tini -- /usr/l..."  8 minutes ago  Up 7 minutes (healthy)  127.0.0.1:5601->5601/tcp
kibana-local-dev
2a33b04ebf92   docker.elastic.co/elasticsearch/elasticsearch:9.3.0  "/bin/tini -- /usr/l..."  8 minutes ago  Up 8 minutes (healthy)  127.0.0.1:9200->9200/tcp, 9300/tcp
es-local-dev
```

AWS CLI

Source: [here](#)

Additional: [here](#)

On your OWN laptop, open CMD as admin and install both AWS CLI and SSM plugin.

Verification: `aws --version`

Session Manager CLI – SSM Plugin

On your OWN laptop, open CMD as admin:

Installation: [here](#)

Verification: [here](#)

Verification: `session-manager-plugin`

AWS Login

In the same terminal, type: `aws login`

This will give you a secure tunnel to access

Then, open PowerShell,

```
aws ssm start-session `
```

```
--target i-035d4cb1342acd867 `
```

```
--document-name AWS-StartPortForwardingSession `
```

```
--parameters "portNumber=5601,localPortNumber=5601" `
```

```
--region us-east-1
```

```
aws ssm start-session `
```

```
--target <YOUR_SOC_INSTANCE_ID> `
```

```
--document-name AWS-StartPortForwardingSession `
```

```
--parameters "portNumber=5601,localPortNumber=5601" `
```

```
--region <YOUR_REGION>
```

You should see something like this after running the script above:

```
Updated profile default to use aws-ssm-agent
PS C:\Users\User> aws ssm start-session
>> --target i-035d4cb1342acd867 `
>> --document-name AWS-StartPortForwardingSession `
>> --parameters "portNumber=5601,localPortNumber=5601" `
>> --region us-east-1

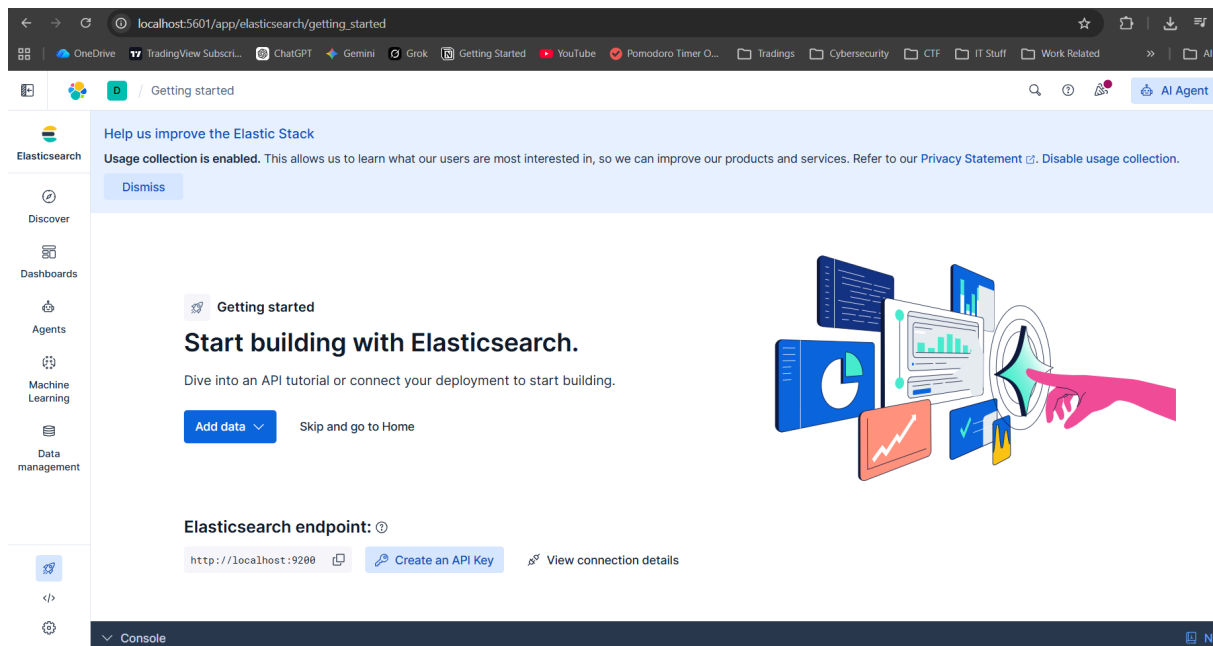
Starting session with SessionId: root-1
Port 5601 opened for sessionId root-1
Waiting for connections...
```

Next, go to your favorite browser and type:

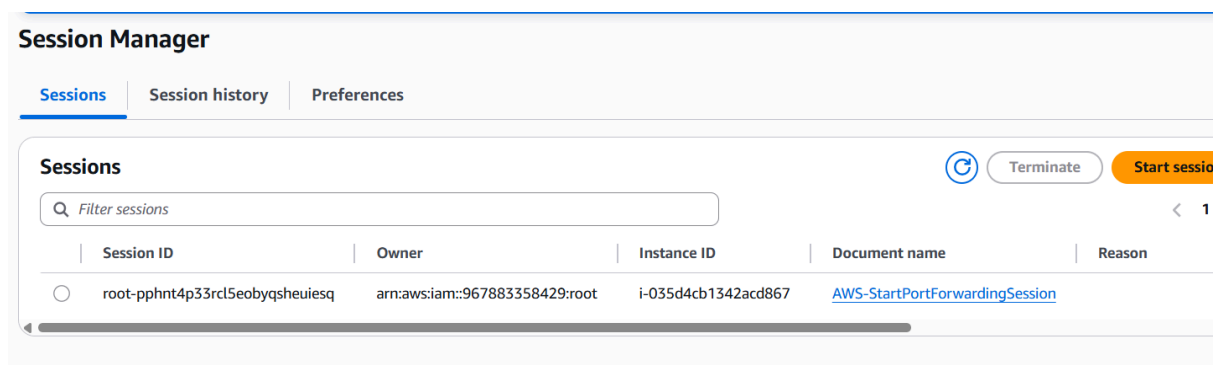
```
http://localhost:5601
```

Enter your credentials from the Elastic Setup – Elastic Credential part at the top of this document.

Result:



You can double-check in the SSM console:



WebUI Elastic Setup

Add Fleet Server

We need to add a fleet server to our own SOC platform instance. Go to the search bar of the web and search for Fleet. Go to Settings.

Step 1: Add the Fleet Server

1. Click that "Add Fleet Server" button.
2. Name: Type SOC-Fleet-Server.
3. URL: This is where you must use your SOC Instance's Private IP (the 10.x.x.x one from your AWS console).
 - Type: `https://<YOUR-SOC-PRIVATE-IP>:8220`

4. Click "Generate Service Token" or "Generate Policy".
5. Check your SOC instance machine's spec because you need to choose the machine type:
 - In your SSM SOC instance, type: `hostnamectl`
 - Look at the Architecture row.

```
$ hostnamectl
Static hostname: ip-10-1-1-25
Icon name: computer-vm
Chassis: vm
Machine ID: ec248801ac8aabf8c0367df6d62487a9
Boot ID: 240e8d7703e8491b845769e43b5808c8
Virtualization: xen
Operating System: Ubuntu 24.04.4 LTS
Kernel: Linux 6.14.0-1018-aws
Architecture: x86_64
Hardware Vendor: Xen
Hardware Model: HVM domU
Firmware Version: 4.11.amazon
Firmware Date: Thu 2006-08-24
Firmware Age: 19y 5month 4w 1d
$
```

6. The Command: Elastic will give you a long command that looks like a curl and `sudo ./elastic-agent install`.
 - Copy this command.

Add a Fleet Server

A Fleet Server is required before you can enroll agents with Fleet. Follow the instructions below to set up a Fleet Server. For more information, see the [Fleet and Elastic Agent Guide](#).

Quick Start | Advanced

✓ **Fleet Server policy created.**
Fleet server policy and service token have been generated. Host configured at <https://10.1.1.25:8220>. You can edit your Fleet Server hosts in [Fleet Settings](#).

2 Install Fleet Server to a centralized host

Install Fleet Server agent on a centralized host so that other hosts you wish to monitor can connect to it. In production, we recommend using one or more dedicated hosts. For additional guidance, see our [installation docs](#).

Linux aarch64 | MacOS aarch64 | DEB aarch64 | RPM aarch64 | ... 1 ▼

```
curl -L -O https://artifacts.elastic.co/downloads/beats/elastic-agent/
tar xzvf elastic-agent-9.3.0-linux-x86_64.tar.gz
cd elastic-agent-9.3.0-linux-x86_64
sudo ./elastic-agent install
```

Example:

```
curl -L -O
```

```
https://artifacts.elastic.co/downloads/beats/elastic-agent/elastic-agent-9.3.0-linux-x86_64.tar.gz
```

```
tar xzvf elastic-agent-9.3.0-linux-x86_64.tar.gz
```

```
cd elastic-agent-9.3.0-linux-x86_64
```

```
sudo ./elastic-agent install \
```

```
--fleet-server-es=http://localhost:9200 \
```

```
--fleet-server-service-token=AAEAAWVsYXN0aWMvZmxlZXQtc2VydMvYl3Rva2VuLTE3NzE2NTE5MDY0MTM6NnBscDRlZzZUS09FVGNiTGv2c1V6UQ \
```

```
--fleet-server-policy=fleet-server-policy \
```

```
--fleet-server-port=8220 \
```

```
--install-servers
```

Add Fleet Agent to the SOC instance

Go back to your SSM SOC instance, make sure you are in the ssm-user or simply type: `cd ~`
Then, paste the CLI from the example above.

What this does: It installs the Elastic Agent on the SOC itself and tells it to act as the "Fleet Server" (the manager). It will take about 1–2 minutes. When it's done, the Kibana screen will display "*Fleet Server connected successfully*" or "*Elastic Agent has been successfully installed*".

Note: Don't forget to type: *Y* for Yes. You will be prompted after entering the script

```
$ cd ~
curl -L -O https://artifacts.elastic.co/downloads/beats/elastic-agent/elastic-agent-9.3.0-linux-x86_64.tar.gz
tar xzvf elastic-agent-9.3.0-linux-x86_64.tar.gz
cd elastic-agent-9.3.0-linux-x86_64
sudo ./elastic-agent install \
  --fleet-server-es=http://localhost:9200 \
  --fleet-server-service-token=AAEAAWVsYXN0aWMvZmxlZXQtc2VydMvYl3Rva2VuLTE3NzE2NTE5MDY0MTM6NnBscDRlZzZUS09FVGNiTGv2c1V6UQ \
  --fleet-server-policy=fleet-server-policy \
  --fleet-server-port=8220 \
  --install-servers % Total      % Received % Xferd Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left   Speed
 76  427M   76  325M   0     0  101M    0  0:00:04  0:00:03  0:00:01  101M
100  427M  100  427M   0     0  104M    0  0:00:04  0:00:04 --:--:--  104M
$ elastic-agent-9.3.0-linux-x86_64/data/elastic-agent-356494/components/lenses/
elastic-agent-9.3.0-linux-x86_64/data/elastic-agent-356494/components/lenses/COPYING
elastic-agent-9.3.0-linux-x86_64/data/elastic-agent-356494/components/lenses/access.aug
elastic-agent-9.3.0-linux-x86_64/data/elastic-agent-356494/components/lenses/activemq_conf.aug
elastic-agent-9.3.0-linux-x86_64/data/elastic-agent-356494/components/lenses/activemq_xml.aug
elastic-agent-9.3.0-linux-x86_64/data/elastic-agent-356494/components/lenses/afs_cellalias.aug
elastic-agent-9.3.0-linux-x86_64/data/elastic-agent-356494/components/lenses/aliases.aug
elastic-agent-9.3.0-linux-x86_64/data/elastic-agent-356494/components/lenses/anaconda.aug
elastic-agent-9.3.0-linux-x86_64/data/elastic-agent-356494/components/lenses/anacron.aug
elastic-agent-9.3.0-linux-x86_64/data/elastic-agent-356494/components/lenses/approx.aug
elastic-agent-9.3.0-linux-x86_64/data/elastic-agent-356494/components/lenses/apt_update_manager.aug
elastic-agent-9.3.0-linux-x86_64/data/elastic-agent-356494/components/lenses/aptcacherngsecurity.aug
elastic-agent-9.3.0-linux-x86_64/data/elastic-agent-356494/components/lenses/aptconf.aug
elastic-agent-9.3.0-linux-x86_64/data/elastic-agent-356494/components/lenses/aptpreferences.aug
```

Once you see **"Fleet Server connected successfully"** or **"Elastic Agent has been successfully installed"** in the SSM SOC terminal, go back to the Elastic WebUI and confirm the connection:

Add a Fleet Server

A Fleet Server is required before you can enroll agents with Fleet. Follow the instructions below to set up a Fleet Server. For more information, see the [Fleet and Elastic Agent Guide](#).

Quick Start

Advanced

```
curl -L -O https://artifacts.elastic.co/downloads/beats/elastic-agent/
tar xzvf elastic-agent-9.3.0-linux-x86_64.tar.gz
cd elastic-agent-9.3.0-linux-x86_64
sudo ./elastic-agent install \
  --fleet-server-es=http://localhost:9200 \
  --fleet-server-service-token=AAEAAWVsYXN0aWMvZmxlZXQtc2VydMvYl3Rva2V
  --fleet-server-policy=fleet-server-policy \
  --fleet-server-port=8220 \
  --install-servers
```

✓

Fleet Server connected

You can now continue enrolling agents with Fleet.

Continue enrolling Elastic Agent

⚙️ Next Step: Update Global Settings (The "Private" Fix)

Now that the server exists, we need to make sure every future agent knows to look for it at the right address.

1. In Fleet, go to the Settings tab (top right).
2. Fleet Server hosts: Click the pencil/edit icon. Make sure it is your private IP: `https://<YOUR-SOC-PRIVATE-IP>:8220`.
3. Outputs (Elasticsearch): Click the pencil/edit icon on the "Default" output.
 - Change `http://localhost:9200` to `https://<YOUR-SOC-PRIVATE-IP>:9200`.
4. Click Save.

Fleet server hosts

Specify the URLs that your agents will use to connect to a Fleet Server. If multiple URLs exist, Fleet will show the first provided URL for enrollment purposes. For more information, see the [Fleet and Elastic Agent Guide](#).

Name	Host URLs	Default	Actions
SOC-Fleet-Server	https://10.1.1.25:8220	✓	

[+ Add Fleet Server](#)

Outputs

Specify where agents will send data.

Name	Type	Hosts	Status	Integration syncing	Default	Actions
SOC-ElasticSearch-Output	Elasticsearch	http://10.1.1.25:9200		N/A	Agent integrations Agent monitoring	

[+ Add output](#)

Check status:

```
sudo systemctl status elastic-agent
```

```
sudo elastic-agent status
```

3. Verify the "Gate" is open (Should see 0.0.0.0 or :::9200)

```
sudo netstat -tulpn | grep 9200
```

Modify Docker Network

It is expected that you get an error saying DEGRADE or 127.0.0.1:9200 from the netstat command. In the WebUI, Fleet – Add Agent page, you are expected to see something like **“Unhealthy”** and **“Add Fleet Server”**.

The reason behind this is that the Docker Elasticsearch network is only allowing localhost or 127.0.0.1:9200 of the SOC instance, and cannot send data to the WebUI. Don't forget that the SOC instance inbound rule should have 2 sources that have the same port of 9200, the SOC Instance Security Group, and the production/main VPC's IP range.

Now, let's fix this issue.

Next, we configure the Docker network in the SOC instance.

```
pwd
```

```
cd /home/ssm-user/elastic-start-local
```

```
docker compose down
```

```
nano docker-compose.yml
```

Then, in service/elasticsearch, change:

```
YAML
ports:
  - "${ES_LOCAL_PORT}:9200" # <--- REMOVED "127.0.0.1:" from the front
environment:
  - discovery.type=single-node
  - network.host=0.0.0.0      # <--- ADD THIS LINE
  - ELASTIC_PASSWORD=${ES_LOCAL_PASSWORD}
  # ... (keep the rest the same)
```

Follow by:

```
docker compose up -d
```



```
sudo systemctl status elastic-agent
```

```
sudo elastic-agent status
```

```
sudo netstat -tulpn | grep 9200
```

```
docker ps
```

These are the results you want:

```
$ docker compose up -d
[+] up 4/4
✓ Network elastic-start-local_default Created
✓ Container es-local-dev Healthy
✓ Container kibana-local-settings Exited
✓ Container kibana-local-dev Created
$ sudo netstat -tulpn | grep 9200
tcp        0      0 0.0.0.0:9200          0.0.0.0:*            LISTEN     88109/docker-proxy
tcp6       0      0 :::9200              :::*                  LISTEN     88114/docker-proxy
$ sudo systemctl restart elastic-agent
$ sudo elastic-agent status
┌ fleet
└─ status: (STARTING)
┌ elastic-agent
└─ status: (HEALTHY) Running
$
```

```
$ docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS
e09cec55845f   docker.elastic.co/kibana/kibana:9.3.0  "/bin/tini -- /usr/l..." 31 minutes ago Up 31 minutes (healthy) 127.0.0.1:5601->5601/tcp
a228ead13510   docker.elastic.co/elasticsearch/elasticsearch:9.3.0  "/bin/tini -- /usr/l..." 31 minutes ago Up 31 minutes (healthy) 0.0.0.0:9200->9200/tcp, [::]:9200->9200/tcp, 9300/tcp
es-local-dev
```

Fleet

Centralized management for Elastic Agents.

[Agents](#) [Agent policies](#) [Enrollment tokens](#) [Uninstall tokens](#) [Data streams](#) [Settings](#)

[Agent activity](#) [Add Fleet Server](#) [Add agent](#)

Filter your data using KQL syntax

Status **5** Tags **1** Agent policy **1** Upgrade available

Showing 1 agent

Healthy **1** Unhealthy **0** Orphaned **0** Updating **0** Offline **0** Inactive **0** Unenrolled **0** Uninstalled **0**

☐	Status	Host	Agent policy	CPU	Memory	Last activity	Version	Actions
☐	Healthy	ip-10-1-1-25	Fleet Server Policy rev. 3	1.20 %	144 MB	24 seconds ago	9.3.0	

Rows per page: 20

< 1 >

Configure Logs

Now, it is time to add agents. Remember, I have CloudWatch, CloudTrail, 2 EC2 machines, RDS - MySQL, and S3.

What we do is hybrid. We use both Agents and AWS Integration in Kibana.

- Agents – For both frontend and backend EC2 machines
- AWS Integration – For CloudWatch, CloudTrail, RDS - MySQL, and S3

Elastic Agents

If your frontend and backend storage volume is 8GB, increase them to at least 10-12 GB storage.

Increase Volume/Storage in EC2 instances:

Source: [here](#)

Official Documentation: [here](#)

First, go to your EC2 and find its volume. Click on the volume and modify volume (Action Button at the top right corner). Adjust the storage > Save.

Second, go to your instance and follow along:

Step 1: Free up space immediately

See what's eating space

```
sudo du -sh /* 2>/dev/null | sort -rh | head -20
```

Clean up the elastic agent tarball you downloaded (it's huge)

```
cd ~
```

```
rm -f elastic-agent-*.tar.gz
```

Clean apt cache

```
sudo apt-get clean
```

Check again

```
df -h
```

Step 2: Fix the partition resize

1. Resize the partition to use the full disk

```
sudo growpart /dev/nvme0n1 1
```

2. Then resize the filesystem to fill the partition

```
sudo resize2fs /dev/nvme0n1p1
```

3. Verify

`df -h` # should now show ~11GB

Step 3: Go to AWS Console

Also check if this EC2's EBS volume was recently resized — if the volume was resized in AWS but the OS wasn't notified, a reboot might be needed first:

`sudo reboot`

Then retry the growpart after reboot. Now both machines storage shouldn't be a problem right now.

NEXT

Go to this path in the browser URL: <http://localhost:5601/app/fleet/agents>

Make sure the server is Healthy. If not, go back to the WebUI Elastic Setup – Modify Docker Network above in this document.

Then, click Add Agent.

Policy Name: `Main-VPC-EC2-Policy`

Click Next or whatever button is next.

Pick the right Linux specs. And copy the script.

Before running, verify that the Frontend/Backend can reach the SOC Fleet server:

`curl http://<SOC-PRIVATE-IP>:8220`

If it times out, add an inbound rule to the SOC Security Group allowing TCP 8220 from the Main VPC CIDR. If successful, you should see something like ***“Client sent an HTTP request to an HTTPS server”***.

Next, for each of the EC2 instances, follow these CLI:

`cd ~`

`{Paste-your-script-from-kibana-add-agents}`

Example script:

```
curl -L -O
https://artifacts.elastic.co/downloads/beats/elastic-agent/elastic-agent-9.3.0+build202602051825-
linux-x86_64.tar.gz
```

```
tar xzvf elastic-agent-9.3.0+build202602051825-linux-x86_64.tar.gz
```

```
cd elastic-agent-9.3.0+build202602051825-linux-x86_64
```

```
sudo ./elastic-agent install --url=https://10.1.1.25:8220
```

```
--enrollment-token=SnQ3LWZwd0JONTVvX1RGOEViOUc6YndTYzI3dS1tSkRWWlktZUZ2SFFlQQ== --insecure
```

Note: Don't forget to type: Y for Yes. You will be prompted after entering the script

From Frontend:

```
ubuntu@ip-10-0-1-252:~$ cd ~
ubuntu@ip-10-0-1-252:~$ pwd
/home/ubuntu
ubuntu@ip-10-0-1-252:~$ curl -L -O https://artifacts.elastic.co/downloads/beats/elastic-agent/elastic-agent-9.3.0+build202602051825-linux-x86_64.tar.gz
tar xzvf elastic-agent-9.3.0+build202602051825-linux-x86_64.tar.gz
cd elastic-agent-9.3.0+build202602051825-linux-x86_64
sudo ./elastic-agent install --url=https://10.1.1.25:8220 --enrollment-token=SnQ3LWZwd0JONTVvX1RGOEViOUc6YndTYzI3dS1tSkRWWlktZUZ2SFFlQQ==
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
           Dload  Upload   Total   Spent    Left   Speed
100  427M  100  427M    0     0  200M      0  0:00:02  0:00:02 --:--:-- 200M
elastic-agent-9.3.0+build202602051825-linux-x86_64/NOTICE.txt
elastic-agent-9.3.0+build202602051825-linux-x86_64/data/elastic-agent-356494/components/osquery-extension.ext
elastic-agent-9.3.0+build202602051825-linux-x86_64/manifest.yaml
elastic-agent-9.3.0+build202602051825-linux-x86_64/data/elastic-agent-356494/components/
elastic-agent-9.3.0+build202602051825-linux-x86_64/data/elastic-agent-356494/components/.build.hash.txt
elastic-agent-9.3.0+build202602051825-linux-x86_64/data/elastic-agent-356494/components/LICENSE.txt
elastic-agent-9.3.0+build202602051825-linux-x86_64/data/elastic-agent-356494/components/NOTICE.pf-elastic-collector.txt
elastic-agent-9.3.0+build202602051825-linux-x86_64/data/elastic-agent-356494/components/NOTICE.pf-elastic-symbolizer.txt
elastic-agent-9.3.0+build202602051825-linux-x86_64/data/elastic-agent-356494/components/NOTICE.pf-host-agent.txt
elastic-agent-9.3.0+build202602051825-linux-x86_64/data/elastic-agent-356494/components/NOTICE.txt
elastic-agent-9.3.0+build202602051825-linux-x86_64/data/elastic-agent-356494/components/README.md
elastic-agent-9.3.0+build202602051825-linux-x86_64/data/elastic-agent-356494/components/apm-server
elastic-agent-9.3.0+build202602051825-linux-x86_64/data/elastic-agent-356494/components/apm-server.spec.yml
elastic-agent-9.3.0+build202602051825-linux-x86_64/data/elastic-agent-356494/components/apm-server.yml
elastic-agent-9.3.0+build202602051825-linux-x86_64/data/elastic-agent-356494/components/bundle.tar.gz
elastic-agent-9.3.0+build202602051825-linux-x86_64/data/elastic-agent-356494/components/checksum.yml
```

Elastic Agent has been successfully installed.

From Backend:

```
ubuntu@ip-10-0-2-37:~$ cd ~
ubuntu@ip-10-0-2-37:~$ curl -L -O https://artifacts.elastic.co/downloads/beats/elastic-agent/elastic-agent-9.3.0+build202602051825-linux-x86_64.tar.gz
tar xzvf elastic-agent-9.3.0+build202602051825-linux-x86_64.tar.gz
cd elastic-agent-9.3.0+build202602051825-linux-x86_64
sudo ./elastic-agent install --url=https://10.1.1.25:8220 --enrollment-token=SnQ3LWZwd0JONTVvX1RGOEViOUc6YndTYzI3dS1tSkRWWlktZUZ2SFFlQQ==
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
           Dload  Upload   Total   Spent    Left   Speed
100  427M  100  427M    0     0  196M      0  0:00:02  0:00:02 --:--:-- 196M
elastic-agent-9.3.0+build202602051825-linux-x86_64/NOTICE.txt
elastic-agent-9.3.0+build202602051825-linux-x86_64/data/elastic-agent-356494/components/osquery-extension.ext
elastic-agent-9.3.0+build202602051825-linux-x86_64/manifest.yaml
elastic-agent-9.3.0+build202602051825-linux-x86_64/data/elastic-agent-356494/components/
elastic-agent-9.3.0+build202602051825-linux-x86_64/data/elastic-agent-356494/components/.build.hash.txt
elastic-agent-9.3.0+build202602051825-linux-x86_64/data/elastic-agent-356494/components/LICENSE.txt
elastic-agent-9.3.0+build202602051825-linux-x86_64/data/elastic-agent-356494/components/NOTICE.pf-elastic-collector.txt
elastic-agent-9.3.0+build202602051825-linux-x86_64/data/elastic-agent-356494/components/NOTICE.pf-elastic-symbolizer.txt
elastic-agent-9.3.0+build202602051825-linux-x86_64/data/elastic-agent-356494/components/NOTICE.pf-host-agent.txt
elastic-agent-9.3.0+build202602051825-linux-x86_64/data/elastic-agent-356494/components/NOTICE.txt
elastic-agent-9.3.0+build202602051825-linux-x86_64/data/elastic-agent-356494/components/README.md
elastic-agent-9.3.0+build202602051825-linux-x86_64/data/elastic-agent-356494/components/apm-server
elastic-agent-9.3.0+build202602051825-linux-x86_64/data/elastic-agent-356494/components/apm-server.spec.yml
elastic-agent-9.3.0+build202602051825-linux-x86_64/data/elastic-agent-356494/components/apm-server.yml
elastic-agent-9.3.0+build202602051825-linux-x86_64/data/elastic-agent-356494/components/bundle.tar.gz
```

Elastic Agent has been successfully installed.

Confirmation – EC2 Agents Results

Fleet

Centralized management for Elastic Agents.

[Agents](#) [Agent policies](#) [Enrollment tokens](#) [Uninstall tokens](#) [Data streams](#) [Settings](#)

[Ingest Overview Metrics](#) [Agent Info Metrics](#)

[Agent activity](#) [Add Fleet](#)

Filter your data using KQL syntax

Status **5** Tags **1** Agent policy **2**

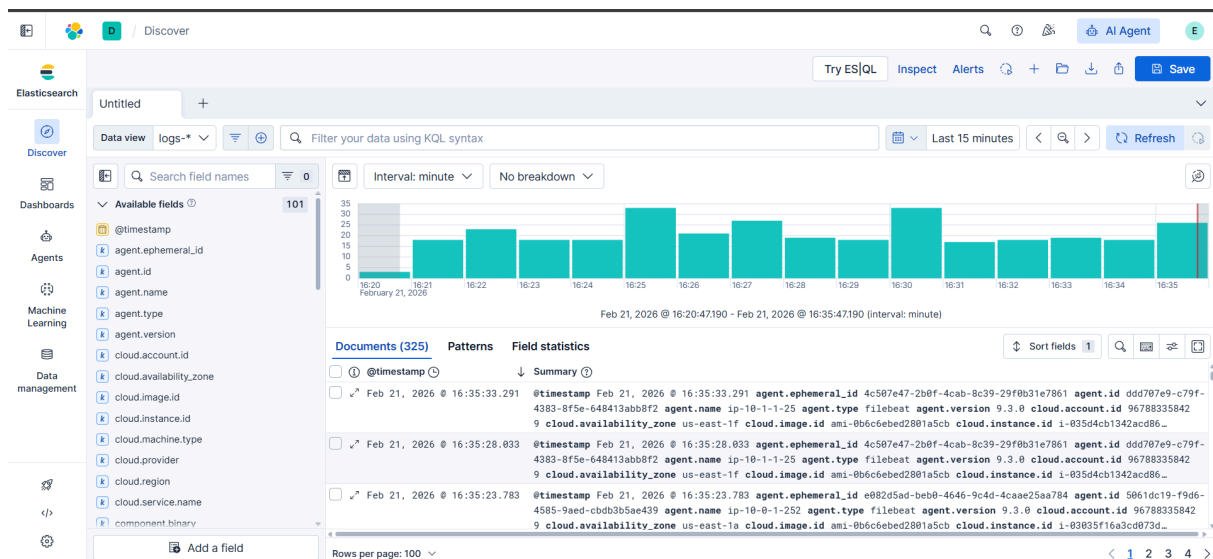
Showing 3 agents

Healthy **3** Unhealthy **0** Orphaned **0** Updating **0** Offline **0** Inactive **0** Unenro

☐	Status	Host	Agent policy	CPU	Memory	Last activity	Version
☐	Healthy	ip-10-0-1-252	Main-VPC-EC2-Policy rev. 1	1.05 %	171 MB	26 seconds ago	9.3.0+build202602051825
☐	Healthy	ip-10-0-2-37	Main-VPC-EC2-Policy rev. 1	1.00 %	173 MB	14 seconds ago	9.3.0+build202602051825
☐	Healthy	ip-10-1-1-25	Fleet Server Policy rev. 3	1.17 %	144 MB	37 seconds ago	9.3.0

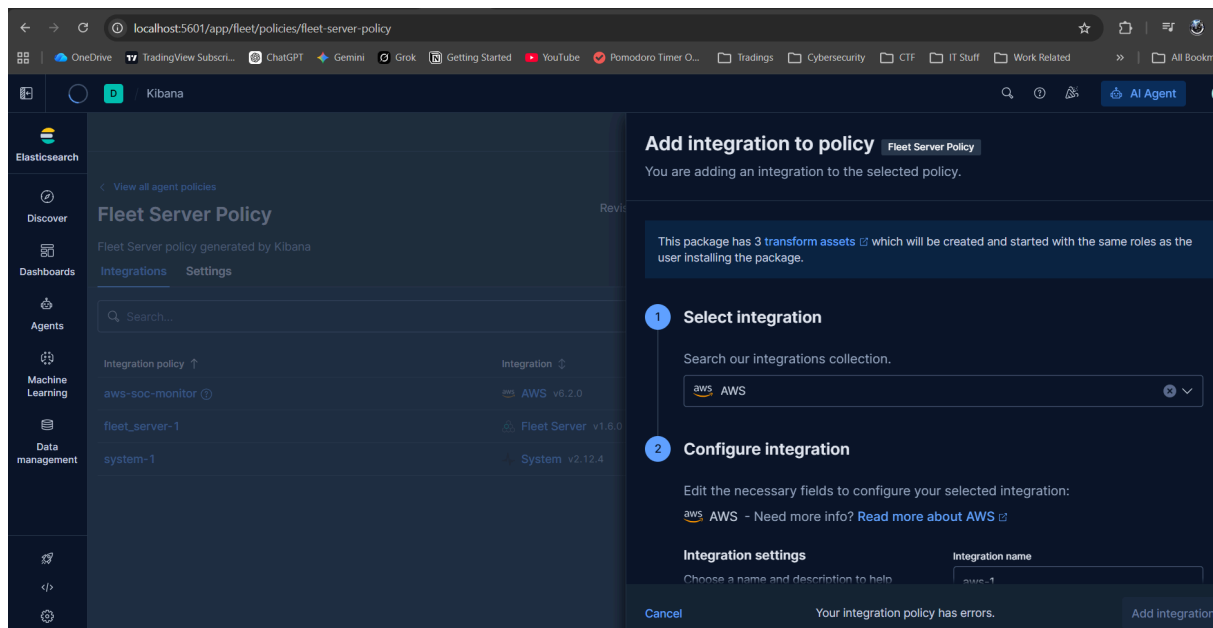
Rows per page: 20

For confirmation, you should look at the left side of the WebUI. Look for Discover and look it. Before you configure agents, this page is empty. Since we already configured agents on both EC2 machines from the Main VPC, we should be able to see logs here. You can read JSON logs within each packet and try to find the machine.



AWS Integration

In this URL: <http://localhost:5601/app/fleet/policies/fleet-server-policy>
You should filter for AWS only.



5 things you should find. Scroll through the list and toggle ON only these specific items:

1. Collect CloudTrail logs from S3 (For your aws-cloudtrail-logs-mj bucket)
2. Collect VPC flow logs from CloudWatch
3. Collect S3 access logs from S3 (For your company-storage-network-project bucket)
4. Collect logs from CloudWatch (For your RDS MySQL Error/Slow logs)
5. Collect RDS metrics (To see database CPU/Memory usage)
6. Collect EC2 metrics (To monitor your SOC/Frontend/Backend hardware health)

NOTE: ONLY USE “*Fleet Server Policy*” when creating any integration. Do not use your non-security platform agent (e.g., from EC2 instances). Example: Main-VPC-EC2-Instance

About Policies

- *Fleet Server Policy* – Is the policy for the agent inside the SOC platform
- *Main-VPC-EC2-Instance* – Is the policy for agents in EC2 hosts. This policy has ‘System Integration’ automatically installed after creation. The ‘System Integration’ is used to track each host’s (including SOC) system metric, such as cpu, ram, storage, etc.

SOC IAM Role - Updated

The purpose of this new IAM role is to make sure the SOC Platform instance is pulling logs from the right place. More importantly, it is supposed to have the right privilege to do so. The code below has *, meaning access to all resources.

However, for "Read-Only" or "Discovery" actions, AWS often requires a wildcard (e.g., ec2:DescribeInstances). The only time it is dangerous is when it is paired with Write or Access actions. (e.g., s3:GetObject on Resource: "*")

Now, we go to the IAM role of the SOC-Private-Instance-Role. We create a new inline policy or modify the old one call "SOC-S3-AccessLogs-Read". Paste this script below.

JSON:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "RestrictedDataAccess",
      "Effect": "Allow",
      "Action": [
        "s3:GetObject",
        "s3:ListBucket",
        "s3:GetBucketLocation"
      ],
      "Resource": [
        "arn:aws:s3:::aws-cloudtrail-logs-mj",
        "arn:aws:s3:::aws-cloudtrail-logs-mj/*",
        "arn:aws:s3:::company-storage-network-project",
        "arn:aws:s3:::company-storage-network-project/*",
        "arn:aws:s3:::company-storage-access-logs",
        "arn:aws:s3:::company-storage-access-logs/*"
      ]
    },
    {
      "Sid": "MandatoryWildcardDiscovery",
      "Effect": "Allow",
```

```

    "Action": [
        "ec2:DescribeInstances",
        "ec2:DescribeRegions",
        "cloudwatch:GetMetricData",
        "cloudwatch:ListMetrics",
        "logs:DescribeLogStreams",
        "rds:DescribeDBInstances",
        "logs:DescribeLogGroups",
        "logs:GetLogEvents",
        "logs:FilterLogEvents"
    ],
    "Resource": "*"
}
]
}

```

Save.

=====

CloudTrail

1. Open Kibana.
2. Click the **Search Bar** at the top and type "**Fleet**". Click it.
3. Click the **Agent Policies** tab.
4. **IMPORTANT:** Click on your **Fleet Server Policy** (The one your SOC instance uses).
 - *Do NOT click "Main-VPC-EC2". If you add it there, the web servers will try to run it again.*

Configuration page:

Under "Configure integration":

- Integration name: cloudtrail-soc (or anything you like)

Under "**Collect CloudTrail logs from S3**":

- Toggle this ON
- Queue URL: Leave blank for now
- Bucket ARN or name: type SOC-Trail-Audit (your bucket name)
- Access Key ID: Leave blank
- Secret Access Key: Leave blank
- Session Token: Leave blank
- Region: us-east-1 (or whatever your region is)

Leaving credentials blank is intentional — it will automatically use your SOC-Final-Role IAM role attached to the instance.

Then click "Add Integration".

For confirmation, you should run this command on 4 places. Make sure it shows HEALTHY.

- Main VPC: Public & Private Instances
- Security VPC: SOC Platform

```

status: (HEALTHY) Running
ubuntu@ip-10-0-2-37:~$ sudo elastic-agent status
fleet
└─ status: (HEALTHY) Connected
elastic-agent
└─ status: (HEALTHY) Running
ubuntu@ip-10-0-2-37:~$

```

☐	Healthy	ip-10-0-1-252	Main-VPC-EC2-Policy rev. 13
☐	Healthy	ip-10-0-2-37	Main-VPC-EC2-Policy rev. 13
☐	Healthy	ip-10-1-1-25	Fleet Server Policy rev. 4

Regardless of the IP or location, just look for the word HEALTHY from the related agents. Should run this every time you set up each integration to see what failed to avoid a huge headache in the future.

VPC Flow Logs

- Find the Toggle: Scroll down to "Collect VPC flow logs from CloudWatch".
- Toggle it: ON.
- Click "Change defaults" and fill in these exact boxes:
 - Log Group Name: Enter the name of the log group you created in AWS (e.g., vpc-flow-logs or /aws/vpc/main).

- Region: Your AWS region (e.g., us-east-1).
- You don't necessarily need to fill in other boxes besides what are shown in this document. Done.
- Additional Info – Scroll down to **"Collect logs from CloudWatch"** (the general one):
 - Note: You can actually use either the specific "VPC flow logs" toggle OR the general "Collect logs" toggle. I recommend using the "Collect VPC flow logs from CloudWatch" one because it has a built-in "parser" that knows how to read the VPC columns (Source IP, Port, Action) automatically.
 - Also, choosing Collect Logs from S3 is better because flow logs output A LOT of data. Plus, CloudWatch increases in price as the volume/storage increases.

S3 Access Logs

This is a two-part setup. Many beginners make the mistake of thinking the Elastic Agent reads the production bucket directly. In reality, S3 Access Logging is a feature you turn on in AWS that "writes" text files about what's happening. Then, the Elastic Agent reads those text files.

First, you need to make another bucket to store logs from S3 Access Logs. You can name it:

`company-storage-access-logs`

Phase 1: The AWS Side (Generate the Logs)

AWS does not log S3 access by default because it costs a tiny bit of storage money. You have to turn the "recorder" on.

1. Go to the S3 Console and click on your production bucket:
company-storage-network-project.
2. Click the Properties tab.
3. Scroll down to Server access logging and click Edit.
4. Select Enable.
5. Target Bucket: Choose a bucket to store the logs. (Earlier, you mentioned company-storage-access-logs. Use that one).
6. Prefix: Type `logs/`
 - Why? This keeps the log files in a neat folder so they don't clutter the root of your bucket.
7. Click Save changes.

Note: It can take up to one hour for AWS to start delivering the first log files to that bucket. Don't panic if Elastic doesn't see anything immediately.

Phase 2: The IAM Side (The Keys)

Your SOC agent needs to be able to "reach into" the access logs bucket, not just the production bucket.

1. Go to your SOC-Private-Instance-Role in IAM.
2. Update your custom policy to ensure the Resource includes the destination bucket where the logs are being written:

```
{  
  "Effect": "Allow",  
  "Action": [  
    "s3:GetObject",  
    "s3:ListBucket",  
    "s3:GetBucketLocation"  
  ],  
  "Resource": [  
    "arn:aws:s3:::company-storage-access-logs",  
    "arn:aws:s3:::company-storage-access-logs/*"  
  ]  
}
```

Phase 3: The Kibana Side (The Toggle)

Now we tell the Elastic Agent to go and fetch those files.

1. Go to Fleet > Agent Policies and click your Fleet Server Policy (SOC).
2. Find your AWS Integration and click Edit.
3. Scroll down to the toggle: Collect S3 access logs from S3.
4. Toggle it ON.
5. Click Change defaults (the blue text) and fill in these three boxes:
 - Bucket Name: company-storage-access-logs (This is the bucket containing the logs).
 - Prefix: `logs/` (Matches what you typed in Phase 1).
 - Region: Your AWS region (e.g., us-east-1).
6. Scroll to the bottom and click Save and Deploy.

RDS Logs – MySQL

RDS is the only service that does not send logs to CloudWatch by default. We have to tell AWS to start "exporting" them.

Step 1: The AWS Side (Enable RDS Log Exports)

1. Log in to the AWS Console and go to RDS > Databases.
2. Click on your Database Name.
3. Click the Modify button at the top right.
4. Scroll down to the Log exports section.
5. Check the boxes for:
 - Error log
 - General log
6. Scroll to the bottom and click Continue.
7. Select Apply immediately and click Modify DB Instance.
 - Note: This does not reboot your database, but it tells AWS to start creating the Log Groups in CloudWatch.

Step 2: Configure RDS Logs in Kibana

Go to Management > Fleet > Agent Policies > Fleet Server Policy and click Edit on your AWS integration.

1. Find the Toggle: Scroll to Collect logs from CloudWatch.
2. Toggle it: ON.
3. Click "Change defaults".
4. Add Error Logs:
 - Log Group Name (not ARN): /aws/rds/instance/<YOUR_DB_NAME>/error
 - Region: Your region (e.g., us-east-1).
5. Add General Logs:
 - Since you want both, look for an "Add row" or "+" button (depending on your Kibana version).
 - If you don't see one, you can simply add the AWS Integration a second time to the same policy specifically for General logs, which I did make a new one called "aws-rds-general-logs-monitor".
 - Log Group Name: /aws/rds/instance/<YOUR_DB_NAME>/general
 - Region: Your region.

 Warning for Beginners: "General logs" record every single SQL query. If your app is busy, this will generate a lot of data (and cost). Keep an eye on it!

Step 3: Enable Metrics (The "Easy" Switches)

While still editing that same AWS integration, look for these two specific toggles:

RDS Metrics (CPU/Memory/Connections)

1. Find the Toggle: Collect RDS metrics.
2. Toggle it: ON.
3. Region: us-east-1a or find it yourself in the RDS console
4. Config: Leave everything at default. Your IAM role's `rds:DescribeDBInstances` permission allows the agent to find the database and pull performance data automatically.

EC2 Metrics (Frontend/Backend/SOC Health)

1. Find the Toggle: Collect EC2 metrics.
2. Toggle it: ON.
3. Config: Leave at default.
4. The Magic: Because your SOC instance has `ec2:DescribeInstances` on Resource: "*", this one agent will now "look" at all three of your servers and report their CPU and RAM usage to Kibana. You do not need to enable this on the Frontend/Backend policies.



Step 4: Final Save

1. Scroll to the bottom.
2. Ensure AWS Credentials (Access Key/Secret) are still EMPTY.
3. Click Save and Deploy.

=====

Confirmation – AWS Integration Results:

These are the policies to be made. The `aws-soc-monitor` is our main integration using the Fleet Server as its Policy.

The `aws-rds-general-logs-monitor` is using the same concepts as the `aws-soc-monitor`. However, it has only 1 selected log, which is Collect logs from CloudWatch – this is to collect logs from MySQL Database – General Logs.

[View all agent policies](#)

Fleet Server Policy

Fleet Server policy generated by Kibana

[Integrations](#)[Settings](#)

Q Search...

Namespace ▾

Add integration

Integration policy ↑	Integration ↕	Namespace	Output	Action
aws-rds-general-logs-monitor	AWS v6.2.0	default ⓘ	fleet-default-output ⓘ	
aws-soc-monitor ⓘ	AWS v6.2.0	default ⓘ	fleet-default-output ⓘ	
fleet_server-1	Fleet Server v1.6.0	default ⓘ	fleet-default-output ⓘ	
system-1	System v2.12.4	default ⓘ	fleet-default-output ⓘ	

=====

END